# Intelligent Systems

Exam Prep

# Disclaimer

- My slides and knowledge are based off last years course content. If I mention something Kurt didn’t talk about this year (or miss something), then please let me know!

# Content

1. Intelligence
2. Expert Systems
3. Agents
4. Multi-Agent Systems
5. Evolutionary Algorithms
6. Reinforcement Learning
7. AlphaGo
8. Multi-Agent Reinforcement Learning

# 1. Intelligence

# Intelligence – What is intelligence?

- **There is no commonly accepted definition of "intelligence"**

- The everyday notion of *"intelligence"* is used as reference / starting point of *"artificial intelligence"* (AI)

- General belief (but not official!):
  - **Capable** (able to do something)
  - **Autonomous** (operate without human intervention)
  - **Believable** (convince humans it is intelligent)

# Intelligence - The Turing Test

- “A system is intelligent if it can answer questions by a person in such a way that the person cannot tell if the answer is given by a human or computer”
- Operational definition (based on distinguishability from human intelligence)
- **Theory of Mind:** ability to reason from someone else’s perspective (a big roadblock for AI, even today)
- The Turing Test is mostly irrelevant now

# Intelligence – Strong vs Weak AI

- Current motivations for AI research are mostly either:
  - **Visionary:** “AI will be able to produce intelligent behaviour”
  - **Pragmatic:** “AI will be able to simulate intelligent behaviour”

- **Strong AI:** capable of thinking / reasoning / understanding, etc.
- **Weak AI:** only pretending to be intelligent

- Strong AI vs Weak AI is often thought about using the **Chinese Room** thought experiment: does a person locked in a room with a Chinese dictionary really understand Chinese? Does ChatGPT understand the answers it gives to you?

# Intelligence – Knowledge-Based AI

- AI is all about “being able to think”
- guiding model = human reasoning capabilities

- **Symbol System Hypothesis:** being able to produce and manipulate “symbols” (a representation of something) is a necessary and sufficient condition for intelligence

- Problem: **Symbol Grounding** (how to make these “symbols” relate to something in the real world)

# Intelligence – Behaviour-Based AI

- Intelligence can emerge from the interaction of simple behaviours (e.g. individual ants are not seen as intelligent, but a whole colony is)

- **Physical Grounding Hypothesis:** symbols being grounded to the real world is a necessary and sufficient condition for intelligence

- Knowledge-based AI: **top-down** design (start with high level concepts, break down into smaller units)
- Behaviour-based AI: **bottom-up** design (start with simple behaviours, combine into more complex behaviours, e.g. Mars Rover lab)

# Intelligence – Connectionism

- Intelligence can be simulated through simple but many interconnected units

- Basically just an artificial neural network, that’s all…

- Key characteristics:
  - **Parallelism**
  - **Sub-symbolic information processing** (i.e. there is no semantic knowledge being communicated, just low-level signals)

# Intelligence – Distributed AI

- AI cannot be simulated through a simple “box”, interacting with other beings is a necessary condition for intelligent behaviour

- Information shared on the knowledge level rather than signal level

- Key characteristics:
  - **Parallelism**
  - **Scalability**
  - **Robustness** (stepping on a single ant doesn’t destroy the whole colony)

# Intelligence – Exam Questions

1. An agent that can classify human-drawn digits using an MLP
   (A) Knowledge-based (B) Behaviour-based (C) Connectionism (D) Distributed AI
2. An NPC in a video game that attacks players on sight
   (A) Knowledge-based (B) Behaviour-based (C) Connectionism (D) Distributed AI
3. A self-driving car that can autonomously navigate traffic
   (A) Knowledge-based (B) Behaviour-based (C) Connectionism (D) Distributed AI
4. An agent that answers FAQs in a call centre
   (A) Knowledge-based (B) Behaviour-based (C) Connectionism (D) Distributed AI

# Intelligence – Exam Questions

1. An agent that can classify human-drawn digits using an MLP

   (A) Knowledge-based (B) Behaviour-based **(C) Connectionism** (D) Distributed AI

   Information is shared on signal level

2. An NPC in a video game that attacks players on sight

   (A) Knowledge-based **(B) Behaviour-based** (C) Connectionism (D) Distributed AI

   *The NPC has multiple behaviours. Default behaviour may be "patrol the area", which is overwritten by "attack" once a player is in sight. The attack behaviour may be overwritten by "reload" if it is out of bullets, etc.*

3. A self-driving car that can autonomously navigate traffic

   (A) Knowledge-based (B) Behaviour-based (C) Connectionism **(D) Distributed AI**

   *A self-driving car has multiple systems that need to work together to make the car drive. These systems might include speed control, lane sensing, traffic awareness, signalling, etc.*

4. An agent that answers FAQs in a call centre

   **(A) Knowledge-based** (B) Behaviour-based (C) Connectionism (D) Distributed AI

# 2. Expert Systems

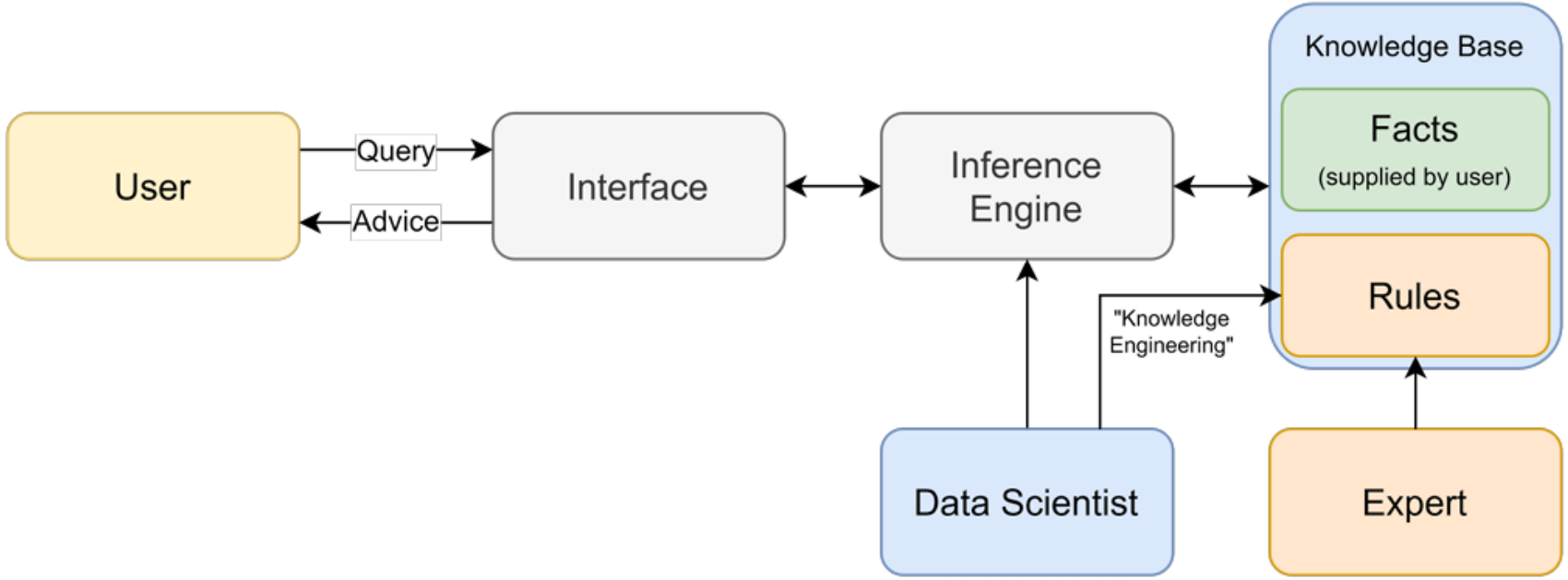
# Expert Systems – What is it?

- A computer system that uses **knowledge** and **inference** to solve problems in a **specialized** domain that are difficult enough to require **human expertise**

- Not really “intelligent”: not proactive, not autonomous…

- Frequently used in hospitals, car garages, other maintenance work, etc. Reason for popularity is that they are **provably correct**

# Expert Systems – Components




Knowledge representation:
- First order logics
- Ontologies
- Non-monotonic logics
- Etc.

The user interacts with the interface to provide the expert system with information, along with a query about the problem. The expert system uses an **inference engine** to reason about the problem, together with the information provided by the user and a predefined set of rules provided by experts. The process of generating these rules from experts is called **knowledge engineering**. Once the expert system has reached a conclusion, it will provide the user with an appropriate advice, along with an explanation of how it constructed this advice.

# Expert Systems – Probabilistic Reasoning

- How to include uncertainty into a model? Often given in **probabilistic facts:**
  e.g. P(rain tomorrow) = 0.6

**Marginalizing** means finding the probability of a certain variable, or collection of variables:

$$P(X_1 = p_1) \text{ or } P(X_1 = p_1, X_2 = p_2, ..., X_n = p_n)$$

**Maximizing** means finding the most probable value of a variable, or collection of variables:

$$X_1 = \underset{p}{\operatorname{argmax}}\, P(X_1 = p)$$

Marginalization is asking "what is the probability of this event happening?", while maximization is asking "what is the most likely event happening?". Note that both of these are computationally expensive, especially when the number of variables grows.

# Expert Systems – Bayes' Theorem

**Bayes' Theorem** is given by the following formula:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

This theorem is used to update the probability of a certain event happening, given new evidence. Another use case of Bayes' Theorem is for classification. For example, given a set of classes $C$ and a set of attributes $a_1, ..., a_n$, what is the most probable class given the measured attributes?

$$\begin{aligned} &\underset{c_i \in C}{\operatorname{argmax}} P(c_i|a_1, ..., a_n) \\ &= \underset{c_i \in C}{\operatorname{argmax}} \frac{P(a_1, ..., a_n|c_i) \cdot P(c_i)}{P(a_1, ..., a_n)} \\ &= \underset{c_i \in C}{\operatorname{argmax}} P(a_1, ..., a_n|c_i) \cdot P(c_i) \end{aligned}$$

The problem with Bayes' Theorem for classification is that we need a lot of data to estimate $P(a_1, ..., a_n|c_i)$, too much to be feasible in practice.

# Expert Systems – Naïve Bayes

Naive Bayes is a simplification of Bayes' Theorem, where we *assume* that the attributes are **conditionally independent**:

$$P(a_1, ..., a_n|c_i) = P(a_1|c_i) \cdot ... \cdot P(a_n|c_i)$$

- Instead of near-exact probabilities, we now maintain a **belief** about the combined probabilities

- Problem: always overestimates probabilities
  - This is fine if we only care about the most probable class, but not if we care about the exact probabilities

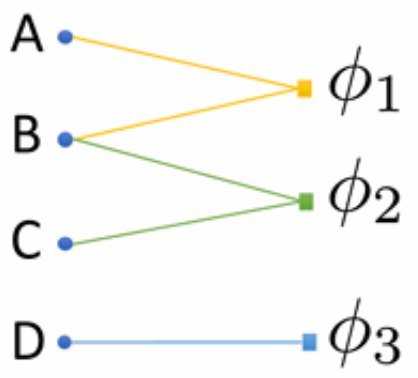
# Expert Systems – Naïve Bayes

- We now have two extremes:
  - **Bayes:** most accurate, but very computationally expensive
  - **Naïve Bayes:** least accurate, very light-weight

- In-between: only make independence assumptions where reasonable

# Expert Systems – Factor Graphs

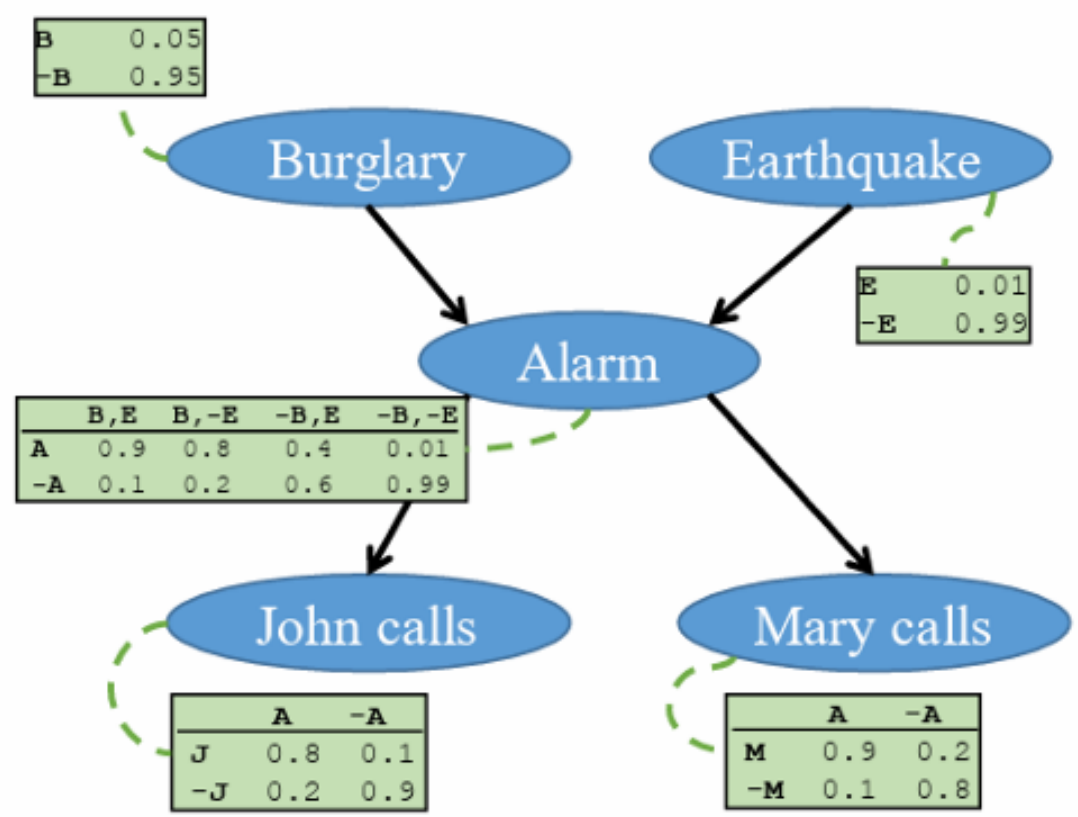
**Factor graphs** group random variables into conditionally independent cliques. Say we have a set of random variables $A, B, C, D$. An example of a factor graph would be:

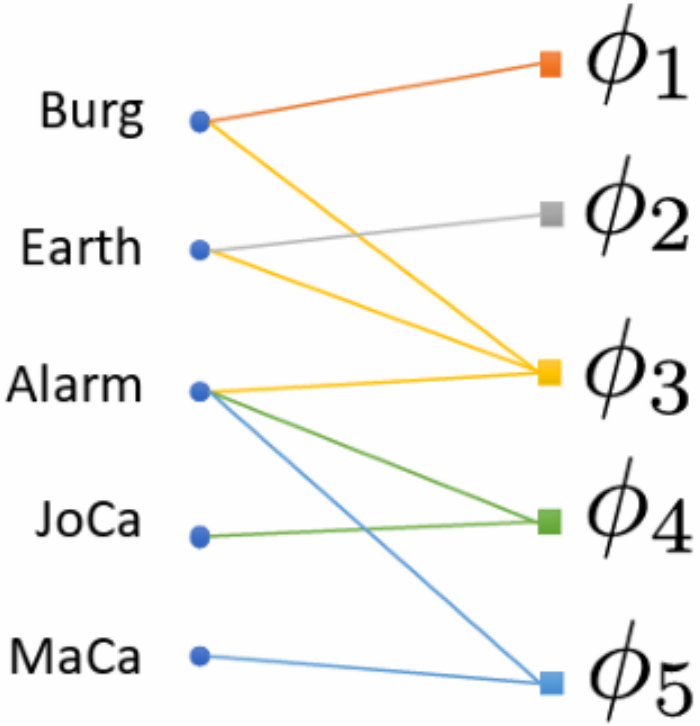



$$P(A, B, C, D) = \frac{1}{Z}\ \phi_1(A, B)\ \phi_2(B, C)\ \phi_3(D)$$

Normalization constant

# Expert Systems – Markov Networks



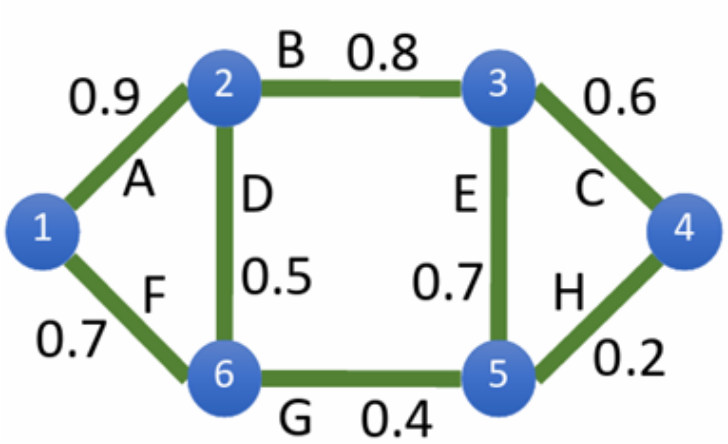



# Expert Systems – Logic and Probability

The current state of the art in expert systems focuses on combining the strengths of logic and probability. This combination is mainly done through **probabilistic predicates**, for example:

$$p : q(t_1, ..., t_n)$$

Here all ground predicates $q(t_1, ..., t_n)$ are true with probability $p$. Logical rules are applied the same way as in classical logic. Here is an example of a program written in ProbLog, that models a graph where some edges might be missing:




```
0.9: edge(1,2)
0.8: edge(2,3)
...
0.4: edge(6,5)
0.2: edge(5,4)
```

Probabilistic part

```
path(X,Y) :- edge(X,Y).
path(X,Y) :- edge(X,Z), path(Z,Y).
```

Logical part

# Expert Systems – Machine Learning

- Another SOTA technique is the combination of expert systems and ML
  - For example, keep rules up-to-date without expert input
- In some fields, e.g. medical imaging, ML has replaced (rule-based) expert systems completely
- Can be combined with LLM, but be careful!
  - Useful LLM technique: **Retrieval-Augmented Generation (RAG)**

# Expert Systems – Summary

- Advantages (compared to human expert):
  - **Consistency** (same input = same output)
  - **Memory** (doesn't forget / misremember information)
  - **Logic** (doesn't make biased decisions)
  - **Infinitely Reproducible** (does not get tired!)

- Disadvantages (compared to human expert):
  - **No common sense** (cannot reason about the world)
  - **No creativity** (cannot come up with new ideas)
  - **Frequent Maintenance** (rules need to be updated)
  - **Not adaptable by itself** (no response to changing conditions)

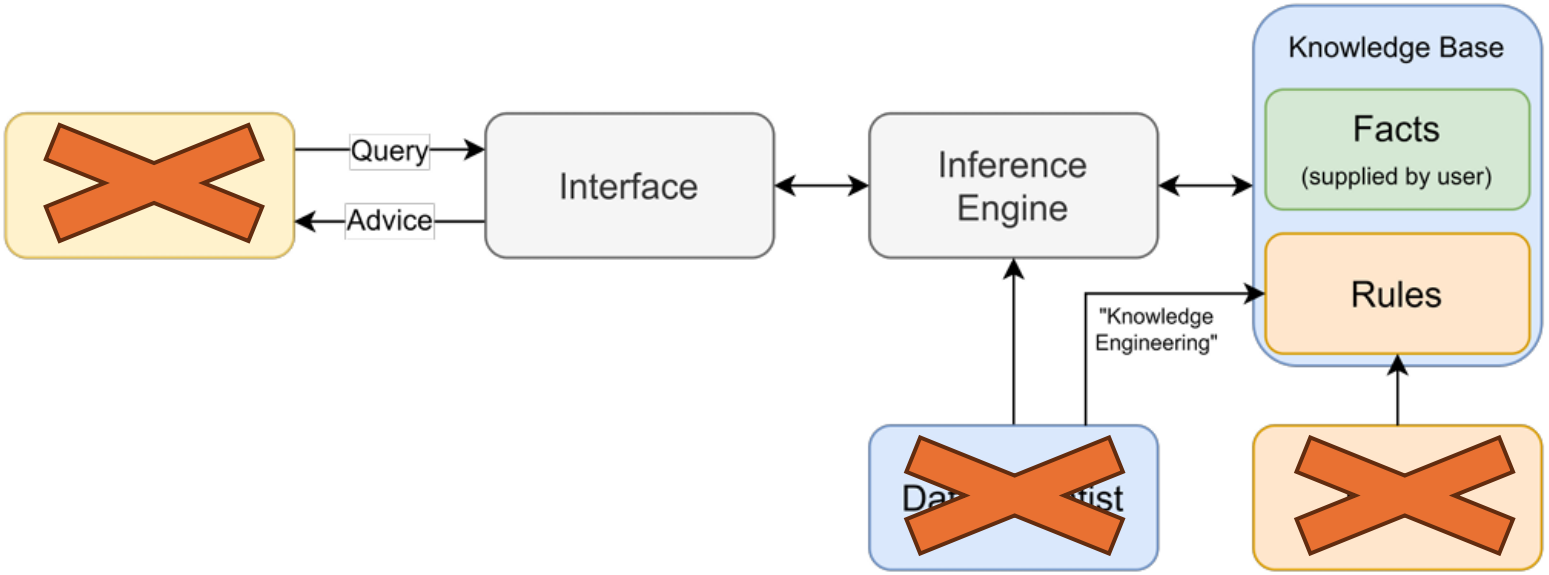
# Expert Systems – Exam Questions

1. What are the main components of an expert system?

2. You are advising a startup that wants to create an advisory tool for people who want to take legal action against online scammers they fell victim to. The startup is unsure if they want their tool to be based on an expert system or an LLM. Give two advantages and two disadvantages for both.

# Expert Systems – Exam Questions

- What are the main components of an expert system?



# Expert Systems – Exam Questions

- You are advising a startup that wants to create an advisory tool for people who want to take legal action against online scammers they fell victim to. The startup is unsure if they want their tool to be based on an expert system or an LLM. Give two advantages and two disadvantages for both.

**Expert Systems Advantages:**
- Provably correct (quite important in legal domain)
- No bias / stereotyping / discrimination
- Law is (mostly) written in a way that can be translated to logic

**Expert Systems Disadvantages:**
- Cannot reason about unique situations
- Will probably not feel very natural to interact with
- Will need more humans in the loop (both in design and operation)

**LLMs Advantages:**
- Can reason about unique situations
- Very natural to interact with
- Easy to design and implement

**LLMs Disadvantages:**
- Might make mistakes
- Can show signs of bias
- Difficult to add new knowledge

*(many more possible answers)*

# 3. Agents

# Agents – What is an agent?

- Similar to intelligence: no commonly accepted definition…

- Emerging standard view: (computational) entity that is **situated** in some environment, capable of **flexible**, **autonomous** activity in order to meet its design objective(s)

- Other characteristics:
  - **Rational** (makes logical decisions based on circumstances)
  - **Mobility** (can move around its environment)
  - **Introspection** (think about what it's doing and why it's doing it)
  - **Benevolence** (being well-meaning / not causing harm)

# Agents – Minimally Intelligent Agent

- A **minimally intelligent agent** is known as the bare minimum of what is needed to call an agent "intelligent":
  - **Pro-activeness** (ability to take initiative)
  - **Reactivity** (ability to perceive and timely respond to environment)
  - **Social Ability** (ability to interact with humans and other agents)

- Big challenge: balancing **goal-directedness** with **reactivity**, hybrid agents must be able to react to new, unseen environments, while also getting things done

# Agents – Environment Characteristics

- **Accessible vs. Inaccessible:** the level of information the agent gets about its environment (complete? acurate? up-to-date?)
- **Deterministic vs. Non-deterministic:** is there a guaranteed effect for actions? is there uncertainty about the next state? Note that sufficiently complex determinism is just as bad as non-determinism.
- **Episodic vs. Non-episodic:** if there are independent stages of behavior (i.e. future is independent of past).
- **Static vs. Dynamic:** if the actions of the agent are the only cause for change in the environment.
- **Discrete vs. Continuous:** if the environment is made up of a finite number of states and actions.

# Agents – Logic-Based Agents

- Based on the idea of knowledge-based AI

"entails"

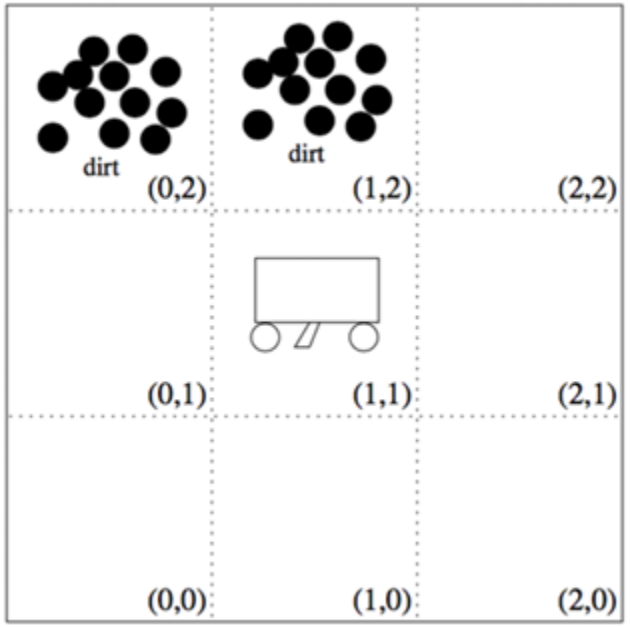
**Belief Database:** all information the agent has about its environment

$$\Delta \vdash_{\rho} \varphi$$

Action chosen by agent

**Theory of Agency:** description of how the agent behaves (in an executable way)

# Agents – Logic-Based Agents




This world can be described using the following symbols:

- $In(x, y)$ := agent is at $(x, y)$.
- $Dirt(x, y)$ := there is dirt at $(x, y)$.
- $Facing(d)$ := the agent is facing in direction $d$.

**Action rules** for logic based agents are defined in the form of "if you can prove this, then do this":

$$\varphi(...) \longrightarrow \psi(...)$$

For the vacuum agent, an example action rule would be:

$$In(x, y) \wedge Dirt(x, y) \longrightarrow Do(suck)$$

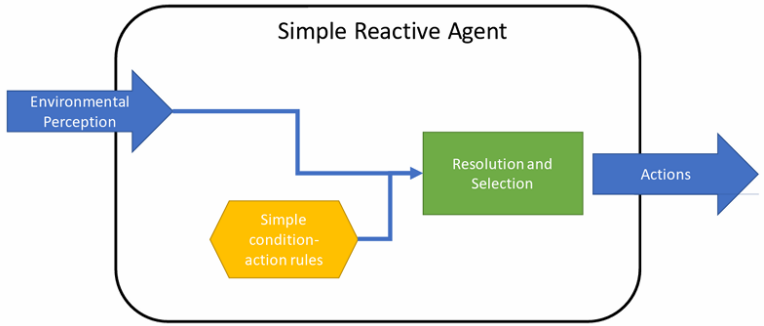
# Agents – Logic-Based Agents

- Problems:
  - **Symbol grounding:** how to relate symbols to the real world
  - **Temporal information:** how to deal with time (e.g. how long should it suck dirt for?)
  - **Complex information:** can’t build a model for everything

- This does not mean logic-based agents are irrelevant!

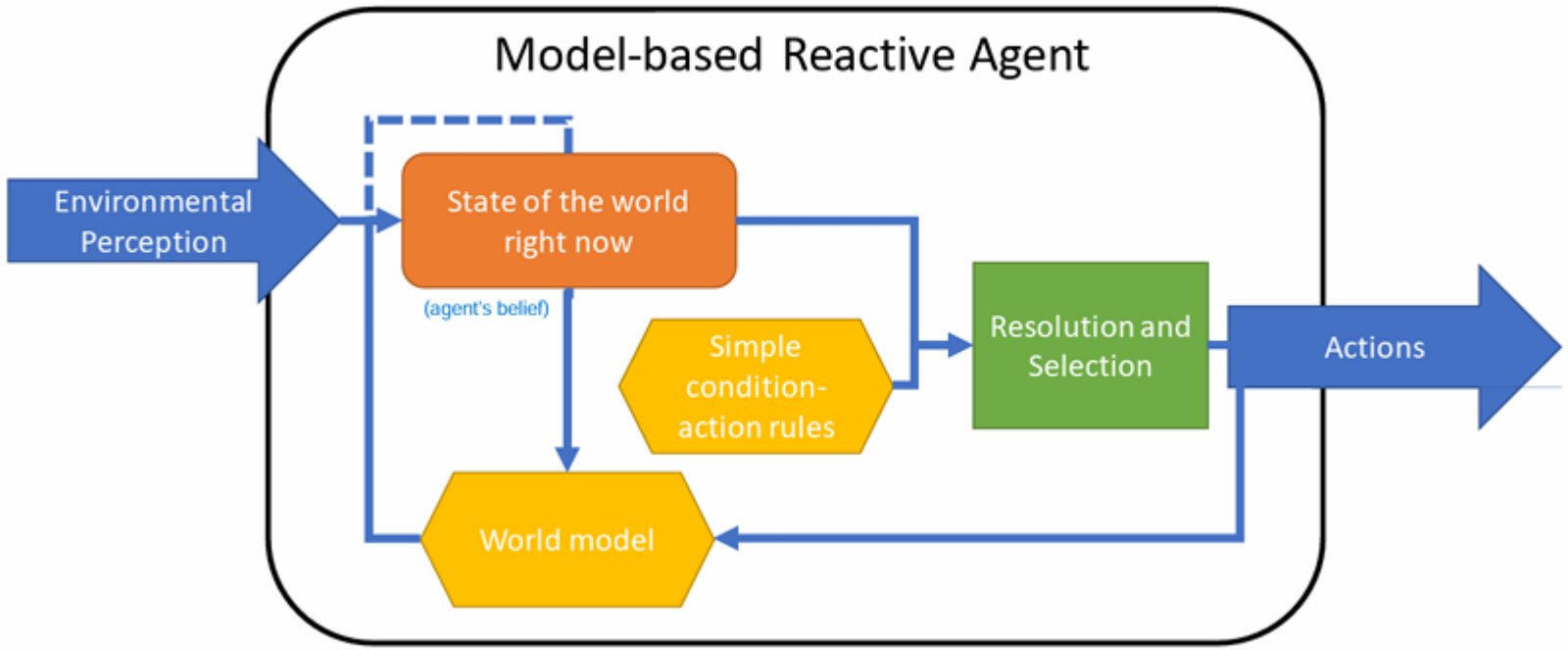
- Logic-based agents start to shine when combined with ML techniques

# Agents – Simple Reactive Agents




- Based on behaviour-based AI
- No complex symbolic reasoning, just a straight mapping from observations to actions
- Conflicting actions? **Subsumption Hierarchy** (think back to Mars Rover lab)
- Advantages: simple, fast, robust, etc.
- Disadvantages: only local information (no model of environment), short term view, emergence hard to predict, dynamics of interactions too complex, etc.

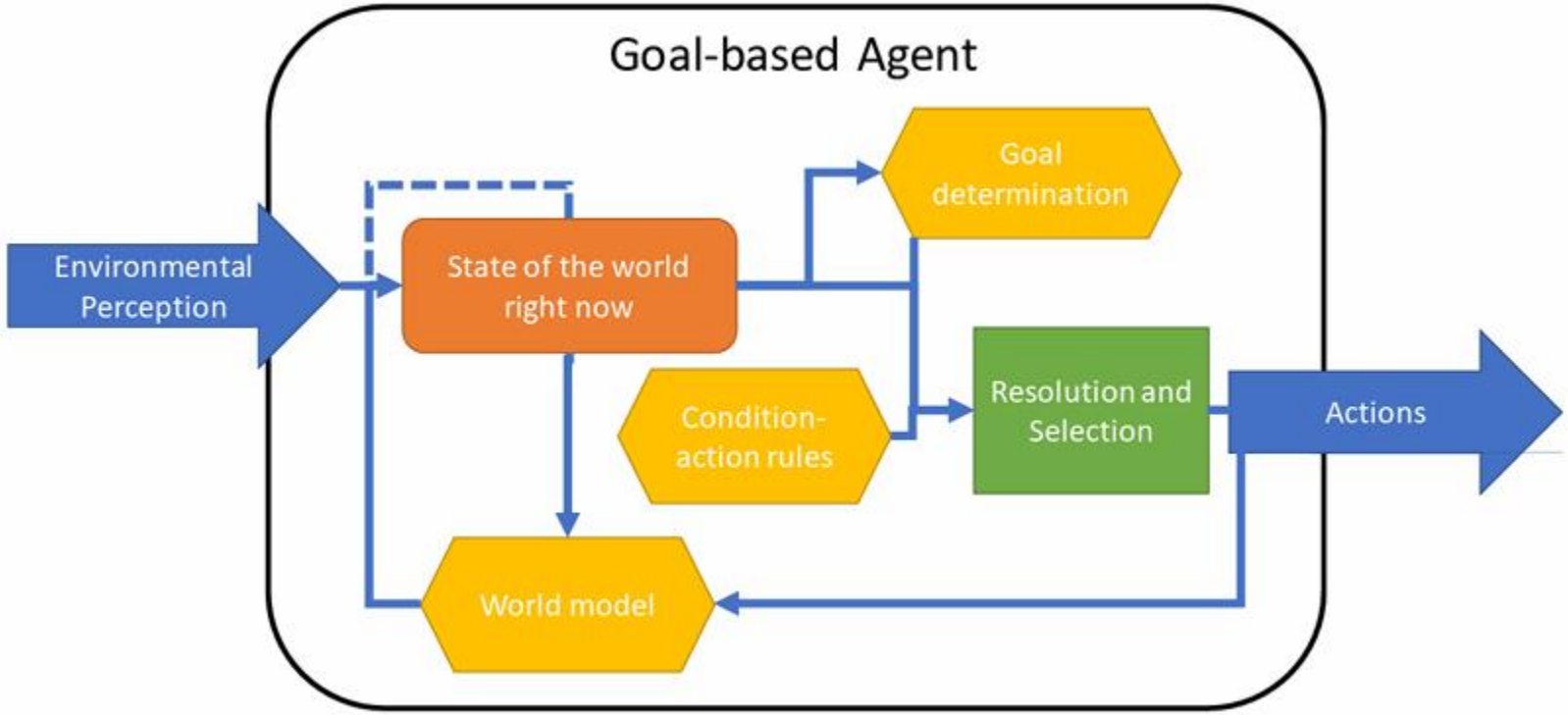
# Agents – Model-Based Reactive Agents




- If environment is not **Markovian** (future independent of past), then the agent maintains a **belief** about the environment, if it is just use RL

# Agents – Goal-Based Agents

- The previous architectures cannot change their goal state depending on the environment, even though this is a key component of intelligent behaviour



# Agents – Goal-Based Agents

- Two types of strategies:
  - **Bold:** bold agents never stop to reconsider their goals
  - **Cautious:** cautious agents constantly stop to reconsider their goals

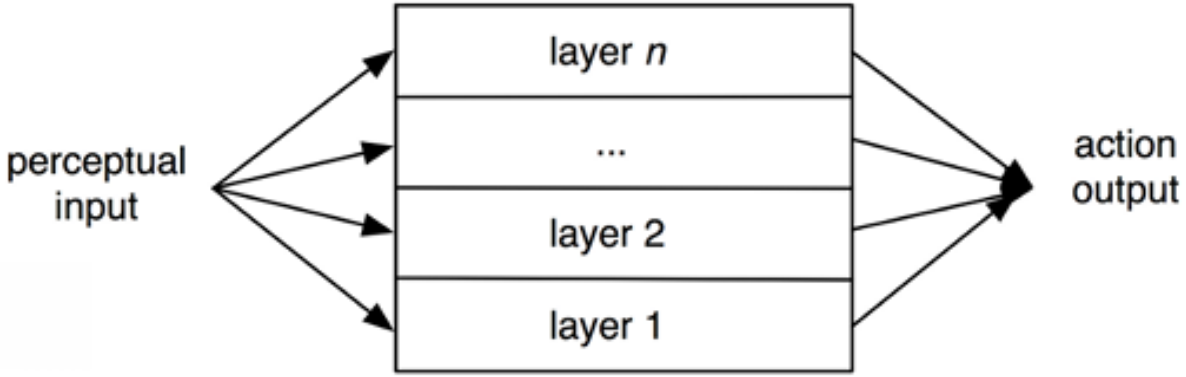
- How to balance goal-directedness with reactivity?

- Implementation: **Layered Architectures**

# Agents – Horizontally-Layered Architecture

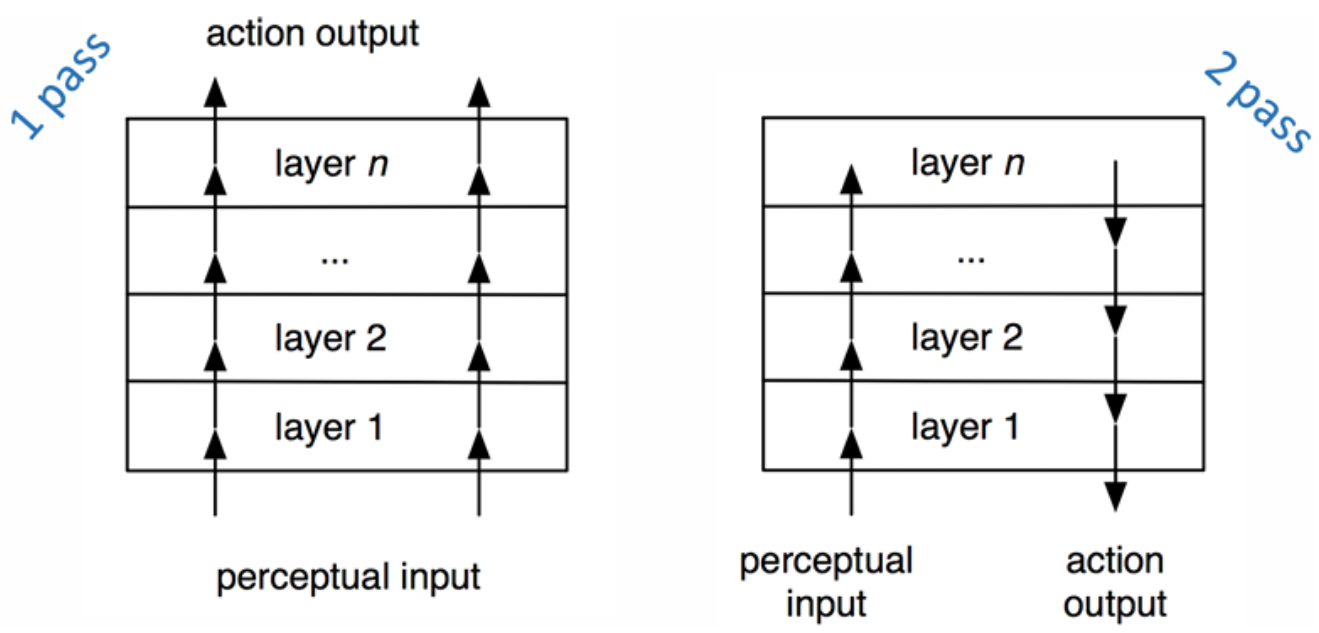
- Number of layers = number of desired behaviours

- Might need a mediator function if behaviours conflict



# Agents – Vertically-Layered Architecture

- Vertically-layered architectures build upon layers of abstraction



# Agents – Exam Questions

1. What problem does a Logic-Based agent have that is solved in Reactive agents?

2. Consider the game of N-Dimensional Chess, where players play chess over an N-dimensional chess board. Both players know N in advance. What are the characteristics of this environment?
   1. Accessible or Inaccessible?
   2. Deterministic or Non-deterministic?
   3. Episodic or Sequential?
   4. Static or Dynamic?
   5. Discrete or Continuous?

3. Should this agent be bold or cautious?

# Agents – Exam Questions

1. What problem does a Logic-Based agent have that is solved in Reactive agents? **No complex symbolic reasoning, just a straight mapping from observations to actions**
2. Consider the game of N-Dimensional Chess, where players play chess over an N-dimensional chess board. Both players know N in advance. What are the characteristics of this environment?
   1. **Accessible** or Inaccessible? *(fully observable for both players)*
   2. **Deterministic** or Non-deterministic? *(no random elements)*
   3. Episodic or **Sequential**? *(able to learn between games)*
   4. Static or **Dynamic**? *(opponent also makes moves)*
   5. **Discrete** or Continuous? *(N is known beforehand, so must be finite)*

3. Should this agent be **bold** or cautious? *(goal is not changing)*

# 4. Multi-Agent Systems

# Multi-Agent Systems – Why?

- Common example: robots moving packages around a warehouse

- Having multiple agents in the same environment brings up some challenges, most notably, **agent coordination**

# Multi-Agent Systems – Coordination

- “Managing dependencies between activities”
- “Special case of interaction in which agents are aware of their dependencies on other agents and adjust accordingly”
- “Any decision that uses information concerning the existence, decisions or decision-making strategies of others”
- Etc.

# Multi-Agent Systems – Coordination

- “Why coordinate? A central control is so much easier to implement…”

- **Bounded Rationality:** “Rationality is limited when individuals make decisions. Under limitations, rational individuals will select a decision that is satisfactory, rather than optimal.”
- Some modern systems are too complex to be controlled by a single entity
- **Distributed Intelligence** (interacting with other beings is a necessary condition for intelligent behaviour)

# Multi-Agent Systems – Coordination

- **Goals:** what are the agents trying to achieve?
- **Activities:** what tasks need to be performed to reach the goals?
- **Actors:** who is responsible for performing the activities?
- **Interdependencies:** how do activities of actors relate to each other?
  - Often assumed to be the most important component

# Multi-Agent Systems – Interdependencies

- **Prerequisites:** output of one activity is required for another
- **Shared Resource:** some resource is required for multiple activities
- **Simultaneity:** multiple activities must be performed at the same time

# Multi-Agent Systems – Client-Server

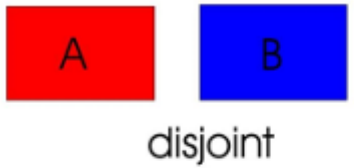
- **Client** sends request for operations to **server**, who is responsible for coordination of its clients

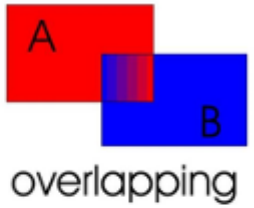
- Client / Server can be dynamically played by agents, or at the same time

- Advantages: easy to implement
- Disadvantages: servers can quickly become bottlenecks, lack of failure tolerance

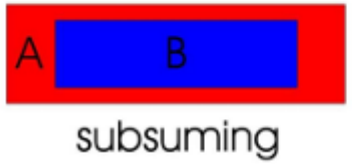
# Multi-Agent Systems – Task-Result Sharing

- Tasks get decomposed into subtasks, which get distributed to agents
- Sub-solutions are combined to form the solution to the original task
- Subtasks: **disjoint**, **overlapping**, **subsuming**, etc.




- Advantages:
  - fewer knowledge and resources required
  - scalable, robust, parallelism, etc.



- Disadvantages: complex to implement, unclear who divides the tasks, how decomposed should the tasks be?

# Multi-Agent Systems – Blackboard

- Agents read and write to a shared memory space
- Operations don’t need to be traceable to a single entity
- Suitable for open source, data- / event-driven applications
- Incremental solution generation

# Multi-Agent Systems – Contract Net

- Network of agents: **managers** and **contractors**
  - **Managers:** announces tasks to be done
  - **Contractors:** bid for the right to perform said task
  - (similarly to client-server, both roles can be dynamically played by agents)

- Middle ground between client-server and blackboard

- Design issues: What tasks should be announced? When should an agent just perform a task instead of announcing it? Who should receive task announcements? Why should contractors bid for tasks?

# Multi-Agent Systems – FA/C

- **Functionally Accurate Cooperation** (FA/C): guideline for cooperation when the individuals' local knowledge is incomplete or inconsistent

- Once noticed that local knowledge is inconsistent, agents should respond by exchanging **functionally correct information** (i.e. reasonable from local pov), and refining this into **global correctness**

- For FA/C to work, all parties must be able to:
  - Measure and evaluate functional correctness
  - Detect inconsistencies between local and global knowledge
  - Revise and extend its local knowledge based on newly obtained information

# Multi-Agent Systems – Joint Planning

- The notion of combining the induvial plans of multiple agents into a single, coordinated plan

- Key design questions:
  - What exactly means a coordinated plan in this environment?
  - How can parties detect (un)coordinatedness?
  - How to jointly generate coordinated plans?

# Multi-Agent Systems – PGP

- **Partial Global Planning** (PGP): general coordination schema with no assumptions about sub-problems, expertise or resources (basically the joint planning version of FA/C)

- Each party can represent and reason about the actions and interactions of other parties and how they affect local activity

- **Partial Global Plan**: specifies how different parts of a plan achieve a common goal

# Multi-Agent Systems – PGP

- Partial Global Plan:
  - **Objective:** reason why the PGP exists
  - **Plan-Activity Map:** what the parties are doing and how it contributes to the objective (includes costs + expected results)
  - **Solution-Construction Graph:** information about how parties should interact, what results should be exchanged, and when to exchange them

- Limitations: local actions executed without a joint agreement, planned coordination is based on very simple abstraction

# Multi-Agent Systems – Exam Questions

1. Two students coordinate the journey from their home to the university by car. This is an example of what interdependency?

   (A) Pre-requisites (B) Shared resource (C) Simultaneity

2. In a warehouse with multiple agents, each agent works with both a global and a local coordinate system. Agents coordinate their goals and actions with other agents in the global coordinate system, then execute them in their local coordinate system. What coordination architecture is this an example of?

   (A) Client-Server (B) Blackboard (C) Contract-Net (D) Partial Global Planning

# Multi-Agent Systems – Exam Questions

1. Two students coordinate the journey from their home to the university by car. This is an example of what interdependency?

   (A) Pre-requisites (B) Shared resource **(C) Simultaneity**

*The goal of the students is to complete the journey from home to university by car. It is a task they must do together. Had the question been "each student needs to go to university, but there is only 1 car" the answer would have been shared resource. In the original question the car is not described as a resource. It is simply part of the action.*

2. In a warehouse with multiple agents, each agent works with both a global and a local coordinate system. Agents coordinate their goals and actions with other agents in the global coordinate system, then execute them in their local coordinate system. What coordination architecture is this an example of?

   (A) Client-Server (B) Blackboard (C) Contract-Net **(D) Partial Global Planning**

# 5. Evolutionary Algorithms

# Evolutionary Algorithms – What is it?

- Solve **optimization problems** using a population of **candidate solutions** and the dynamics of **genetic evolution**

- **Any-time algorithms:** just let it run forever and grab a solution whenever you want

- Iterative process, relies heavily on stochasticity

# Evolutionary Algorithms – Steps

1. Create initial population

2. Select individuals for reproduction

3. Generate new population from reproduction pool

*(2 and 3 are repeated until a stopping criterion is met)*

# Evolutionary Algorithms – Initial Population

- Usually random solutions, can be seeded with domain knowledge for faster convergence

- Needs to be **large** and **diverse**

- **Genotype:** representation of solution (e.g. bitstring)
- **Phenotype:** the solution the genotype encodes (can be evaluated!)

# Evolutionary Algorithms – Selection

- Selection criterion: **fitness function** (aka, the function it tries to optimize)
- Complexity of selection procedure ≈ Complexity of whole algorithm (i.e. it limits population size and number of iterations)
- Possible speed-up: **sampling** (perform selection on subset of population, then generalize)
- Fitness-based selection steers the probability of survival for an individual

# Evo. Algorithms – Roulette Wheel Selection

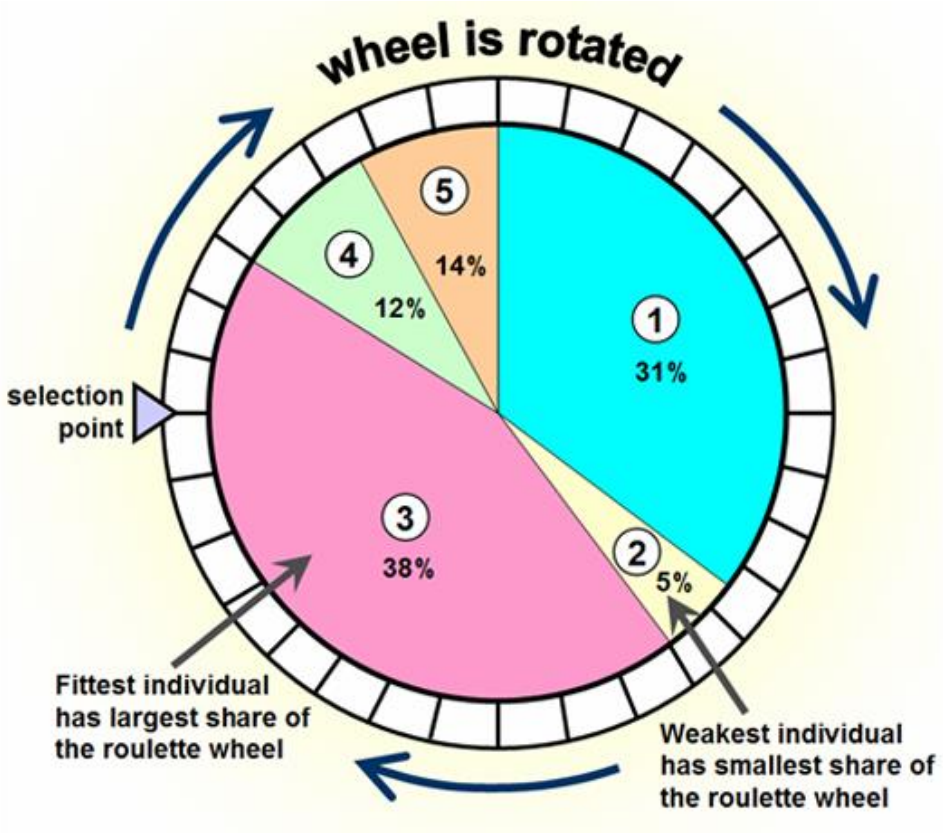
- Probability of being selected is proportional to fitness

Prob. usually given by:

$$p_i = \frac{f_i}{\sum_j f_j}$$

Other option: **Boltzmann dist.**
High temp. = more random
Low temp. = more deterministic

$$p_i = \frac{e^{f_i/T}}{\sum_j e^{f_j/T}}$$



# Evo. Algorithms – Tournament Selection

- Hold several limited-size tournaments, winner(s) get added to reproduction pool

- Smaller tournaments = more diversity, slower convergence

- **Stochastic Tournament Outcomes:** select i'th best individual with prob.

$$p_i = p\,(1-p)^{i-1}$$

# Evolutionary Algorithms – Reproduction

- Create a new population from the reproduction pool

- **Crossover:** combining two genotypes to create a new genotype

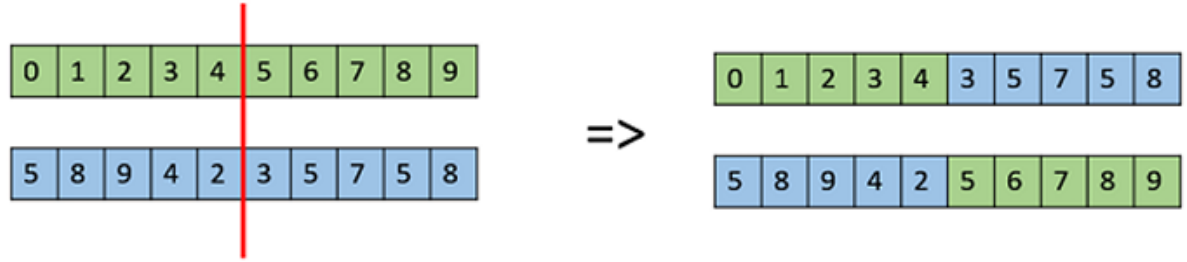
- **Mutation:** change a genotype in a random way

- These operations can be destructive to good solutions
- Solution: **Elitism**: best *k* solutions will always be included in next generation

# Evolutionary Algorithms – Crossover

- **Crossover:** combining two genotypes to create a new genotype
  - **Single-Point Crossover** (most popular)
  - **Multi-Point Crossover**
  - **Headless-Chicken Crossover**

    (population member + random genotype)
  - **Uniform Crossover**
  - Etc.



Uniform Crossover

# Evolutionary Algorithms – Mutation

- **Mutation:** change a genotype in a random way
- Keep mutation rate as low as possible while still increasing diversity!
- **Bit-Flip Mutation** (most common)
- **Real-Value Mutation:** add some Gaussian noise to the genotype (most common if genotype is not binary)

# Evolutionary Algorithms – Schema Theory

- Offers insights into why good solutions survive and become even better

- Consider genotypes:
  - 0110
  - 1110
  - 1100
  - 0100
- All belong to the schema: *1*0 (where * indicates positions where the exact value does not matter)

# Evolutionary Algorithms – Schema Theory

Consider fixed-length schemata of the form:

$$s \in \{0, 1, *\}^n$$

The **order** $o(s)$ is the number of fixed positions in $s$. The **defining length** $d(s)$ is the longest distance between fixed positions in $s$. We can use these definitions to reason about the disruptive properties of mutation and crossover.

# Evolutionary Algorithms – Schema Theory

The probability of a schema to survive mutation is given by:

$$(1 - p_m)^{o(s)}$$

Where $p_m$ is the probability of mutation for a single bit. We can deduce from this that schemata with a higher order have a lower probability of surviving mutation.

The probability of a schema to survive single-point crossover is given by:

$$1 - p_c \frac{d(s)}{n-1}$$

Where $p_c$ is the probability of crossover. We can deduce from this that schemata with a higher defining length have a lower probability of surviving crossover.

# Evolutionary Algorithms – Schema Theory

- Of the schemata with high fitness, those with **low order** and **short defining length** are more likely to survive, this is called the **schema theorem**

- Generalizing to evolutionary algorithms, we can say that evolutionary algorithms work by identifying and combining **building blocks**

- This gives us some guidelines for designing good genotypes:
  - Put related genes close to each other, so that they can form schemata with short defining length
  - Make each gene as independent of other genes as possible
  - Give crossover operator a low probability of destroying schemas

# Evolutionary Algorithms – SNES

- **Separable Natural Evolution Strategy** (SNES)
- Evolutionary algorithm that maintains a Gaussian distribution for each gene

- Individual: sample each gene from its distribution (only works if each gene is independent!)

- **No selection or reproduction**: distributions are updated through a gradient of the population moving in the direction of the fittest individuals
- **No mutation**: resample the population each generation (ensures diversity / prevents suboptimal convergence)

# Evolutionary Algorithms – Neuro-Evolution

- Use evolutionary algorithms to train artificial neural networks

- Genotypes can represent many things:
  - Weights
  - Number of processing units
  - Topology
  - Learning Rate
  - Etc.

- To compute fitness of a genotype, convert it into its phenotype neural network, then evaluate it on the actual task (can be both training / testing)

# Evolutionary Algorithms – Niche Solutions

- A niche solution can draw a lot of the population towards a suboptimal solution

- If too many individuals become similar, crossover does not result in any more exploration, only mutation (which is slow) can move the population out of a suboptimal solution

- To avoid overpopulation of similar individuals, apply a **niche penalty** to a group of sufficiently similar individuals (**niche radius**)

# Evolutionary Algorithms – Problems

- Evolutionary algorithms offer no “guarantee” about the quality of the solution
- Evolutionary algorithms don’t scale well with problem complexity
  - Building block theory suggests that the problem should be divisible into smaller partial solutions, but for many complex problems this isn’t possible
- Designing fitness function can be a challenge
  - Fitness function must be able to separate partial-solutions from non-solutions, with a sufficient spread in fitness values (i.e. evolutionary algorithms cannot solve binary problems that are either solved or not-solved)

# Evolutionary Algorithms – Exam Questions

1. Consider two GA both using Boltzmann selection with a non-decaying temperature: algorithm A uses a temperature of 0.8, algorithm B uses a temperature of 0.6. Which system do you expect to converge faster?

2. Design an evolutionary algorithm for solving the 0/1 knapsack problem that doesn't use bitstrings
   - Genotype:
   - Fitness function:
   - Mutation operator:
   - Crossover operator:

3. Give the order and defining length of the following schemata: 0*1***10*

# Evolutionary Algorithms – Exam Questions

1. Consider two GA both using Boltzmann selection with a non-decaying temperature: algorithm A uses a temperature of 0.8, algorithm B uses a temperature of 0.6. Which system do you expect to converge faster? **B**

2. Design an evolutionary algorithm for solving the 0/1 knapsack problem that doesn't use bitstrings *(multiple solutions possible)*
   - Genotype: **a list of real numbers, representing the "rank" of each item**
   - Fitness function: **construct phenotype by adding items from highest rank to lowest, until capacity is full. Fitness is equal to value**
   - Mutation operator: **add Gaussian noise to the rank of a random item**
   - Crossover operator: **regular single/multi-point crossover**

3. Give the order and defining length of the following schemata: 0*1***10*

**order = 4, defining length = 7**

# 6. Reinforcement Learning

# Disclaimer

- This part has changed a lot compared to last year
- My slides are still based on my notes from last year
- If I miss anything or something is not clear, please let me know

# Reinforcement Learning – What is it?

- Image teaching a robot how to juggle
  - Supervised learning: give robot dataset of juggling movements, learn from that
- In supervised learning you need to know the solution to the problem you are trying to solve beforehand…
- Better solution: just give the robot the balls and let it try!
- **Reinforcement Learning:** learn from rewards / punishments
  - No cognitive interaction required; like teaching a dog how to behave

# Reinforcement Learning – What is it?

- Goal: learn a **policy** that says what action to take in a state

- Key elements:
  - Agents should learn from interactions with environment (not examples)
  - Agents receive delayed rewards (not immediate feedback)
  - Agents should balance **exploration** with **exploitation**

# Reinforcement Learning – MDP

- A single agent reinforcement learning problem can be defined as a **Markov Decision Process** (MDP):
  - $S$: set of states.
  - $A$: set of actions.
  - $\delta : S \times A \to S$: transition function (often unknown).
  - $r : S \times A \to \mathbb{R}$: reward function (often unknown).
  - $\gamma$: discount factor (how much future rewards are worth).

Given this Markov decision process, the goal of the agent is to find a policy $\pi : S \to A$ that maximizes the **value function**:

$$V^{\pi}(s_t) = \sum_{i=0}^{\infty} \gamma^i r_{t+1}$$

Sum of all future rewards starting from state s at time t, assuming all future actions are choosing according to $\pi$

# Reinforcement Learning – Discount Factor

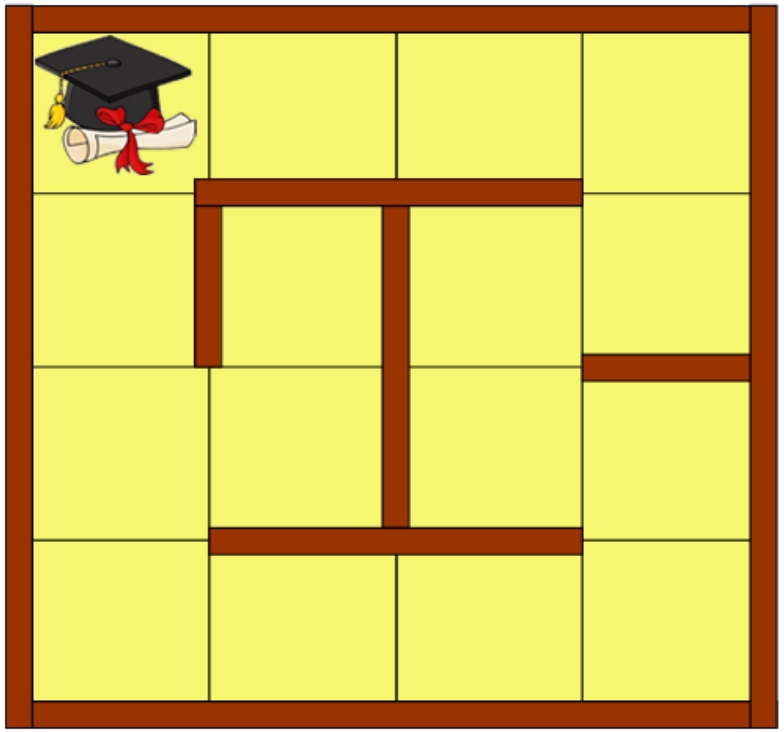
- The discount factor balances immediate rewards with future rewards

$$r_t = \gamma r_{t+1}$$

1. By restricting $\gamma$ to $[0, 1)$, we ensure that $V^\pi(s_t)$ will be finite.
2. By giving more value to immediate rewards, we put some time pressure on the agent (an agent with $\gamma = 1$ will be fine with taking a longer path to a better reward, rather than taking immediate rewards).

# Reinforcement Learning - Example

## An example



State = which square the agent is in

Action = moving up/down/left/right

Reward = 0 for moving anywhere but the top left square, -5 for hitting a wall, +1 for reaching the top left square

Transition = moving to square in move direction unless obstructed by wall

$\gamma$ => getting to the diploma as fast as possible

# Reinforcement Learning – Nondeterminism

- Nondeterminism: Same action in same state might lead to different outcome

$P^a_{ss'} : S \times A \times S \to [0,1]$: transition function (where $s$ is the current state and $s'$ is the follow-up state).

- Can we still learn a policy? Yes, as long as the probabilities are **stationary**
  - i.e. they do not change during the learning process
- Useful for both (non)deterministic environments: **Probabilistic Policy:**

$$\pi(a|s) : S \times A \to [0,1]$$

- Probability of choosing action *a* in state *s*
- Can be used to balance exploration and exploitation

# Reinforcement Learning – Bellman Equation

- Ok so $V^{\pi}(s_t) = \sum_{i=0}^{\infty} \gamma^i r_{t+1}$ gives us the value of a state, what's the value of the policy?

- **Bellman Equation**(s)**:**

$$V(s) = r(s, \pi(s)) + \gamma \sum_{s' \in S} P_{\delta}(s, \pi(s), s') V(s'), \text{ for stochastic environments.}$$

$$V(s) = \sum_{a \in A} P_{\pi}(a|s) \left[ r(s,a) + \gamma \sum_{s' \in S} P_{\delta}(s, a, s') V(s') \right], \text{ for stochastic policies.}$$

- This is the **update rule** of **policy iteration**

# Reinforcement Learning – Policy Iteration

Give each state a random value

Max. change in value in previous full backup

Update value of each state (**full backup**)

**Algorithm 1** Policy Iteration

Initialize $V_0$ arbitrarily.
$t = 1$;
**while** (bellman_error $> \Delta$) **do**
  **for** $s \in S$ **do**
    $V_t^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s' \in S} P_\delta(s, \pi(s), s') V_{t-1}^\pi(s')$
  **end for**
  $t = t + 1$;
**end while**

# Reinforcement Learning – Policy Improvement

- Having a value is nice, but what we really want is the policy with the *highest* value

- Define new policy: select action that maximizes value in new state

$$\pi_t(s) = \underset{a}{\operatorname{argmax}} \left[ r(s,a) + \gamma \sum_{s' \in S} P_\delta(s,a,s') V^{\pi_{t-1}}(s') \right]$$

- Policy improvement + Policy Iteration = **Actor-Critic**

# Reinforcement Learning – Actor-Critic

**Algorithm 2** Actor-Critic Policy Iteration

```
Initialize π_0 and V_0 randomly.
t = 1;
while the values are still changing do
    Update the value of the states using the value update rules.    ▷ Critic
    Update the policy using the policy improvement rule.            ▷ Actor
    t = t + 1;
end while
```

Note: Actor-critic is **guaranteed** to converge to an optimal policy

$$\forall s \in S : V^{\pi_t}(s) \geq V^{\pi_{t-1}}(s)$$

# Reinforcement Learning – Model-Free RL

- Actor-Critic is nice, but it requires in-advance knowledge of the transition and reward function (*very* often not the case in practise)

- We prefer to just let the agent learn from experience!

- Knowing the functions beforehand: **model-based reinforcement learning**
- Going in blind: **model-free reinforcement learning**

- From here onwards, all methods will be **model-free**

# Reinforcement Learning – Monte-Carlo Est.

The idea behind **Monte-Carlo estimation** is to estimate the value of a state by averaging the rewards of all episodes that pass through that state. This essentially comes down to updating a running average of the value of a state, with the following update rule:

$$\tilde{V}(s) = \tilde{V}(s) + \alpha \left( V_{MC}(s) - \tilde{V}(s) \right)$$

Where $\alpha$ is the learning rate, and $V_{MC}(s)$ is the value of the state as estimated by Monte-Carlo estimation.

# Reinforcement Learning – Bootstrapping

In Monte-Carlo estimation, we wait until the end of an episode to update the value of a state. In **bootstrapping**, we only perform a single step of the policy and calculate the **temporal difference error (TD-error)** of the previous state and the current state:

$$\tilde{V}(s) = \tilde{V}(s) + \alpha \left( r + \gamma \tilde{V}(s') - \tilde{V}(s) \right)$$

# Reinforcement Learning – TD-Error

- Very important concept in RL! **“Temporal Difference Error”** – Foundation of human / animal learning

- Difference between what the agent expected based on its experience vs. what it ended up getting

If you learn a monkey that they will receive a piece of banana if they press a button, and then give them a piece of banana without them pressing the button, they will experience a (positive) temporal difference error. On the contrary, if they press the button and you don’t give them a piece of banana, they will experience a (negative) temporal difference error.

# Reinforcement Learning – Q-Learning

- The most popular model-free reinforcement learning algorithm
- Learn the value of every state-action pair:

$$Q(s,a) = r(s,a) + \gamma \sum_{s' \in S} P_\delta(s,a,s') V^{\pi_{t-1}}(s')$$

- After travelling through a *(s, a)* pair, follow the policy and update the Q-value:

$$Q^{\pi_t}(s,a) = r(s,a) + \sum_{i=1}^{N} \gamma^i r(s_i, \pi_t(s_i)) = \sum_{i=0} \gamma^i r_i$$

- Policy chooses action that maximizes the Q-value:

$$\pi_t(s) = \operatorname*{argmax}_a Q^{\pi_{t-1}}(s,a)$$

# Reinforcement Learning – Q-Learning

**Algorithm 3** Q-Learning

```
Initialize Q arbitrarily.
Start in state s.
while true do
    Choose an action a.
    Execute a and observe r and s'.
    Q(s, a) = Q(s, a) + α [r + γ max_a' Q(s', a') − Q(s, a)]
    s = s'
end while
```

Q-learning is **guaranteed** to converge to an optimal policy
(as long as all state-action pairs are visited often enough!)

Learning rate $\alpha$ should be small enough to allow the agent to explore (often *1/visits(s,a)*)

# Reinforcement Learning – GLIE

- Why balance exploration with exploitation?
- To learn a good policy, the whole environment needs to be explored, but to sample good values, the agent should execute the best policy…

- Goal: **Greedy in the Limit with Infinite Exploration (GLIE)**
  - Given infinite time, all state-action pairs are visited infinitely often
  - Given infinite time, the policy converges to a greedy policy

- Two main ways to achieve this: **ε-Greedy** or **Boltzmann Exploration**

# Reinforcement Learning – GLIE

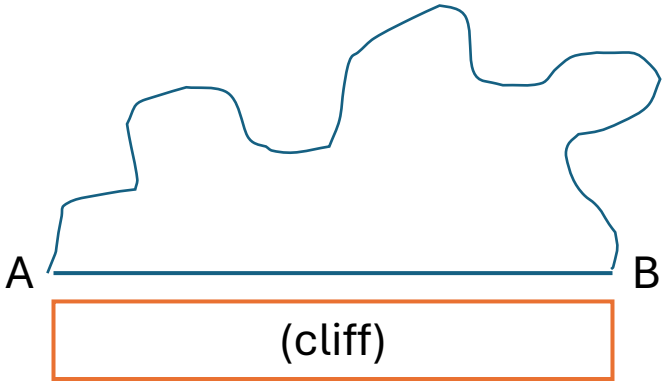
- **ε-Greedy:** with probability ε select a random action, otherwise select the best action
  - GLIE if ε approaches 0 at a rate of *1/t*

- **Boltzmann Exploration:** select action with probability according to a Boltzmann distribution
  - GLIE if *T* approaches 0 at a rate of *1/ln(t)*

$$P(a|s) = \frac{e^{Q(s,a)/T}}{\sum_{a'} e^{Q(s,a')/T}}$$

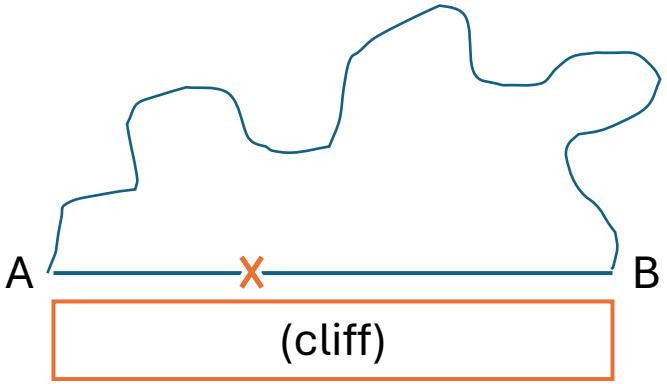
# Reinforcement Learning – Exploration Issue

- Imagine we have a robot that needs to learn how to get from A to B
- The optimal path from A to B to walk along the edge of a cliff, but there is also a longer path that is more safe



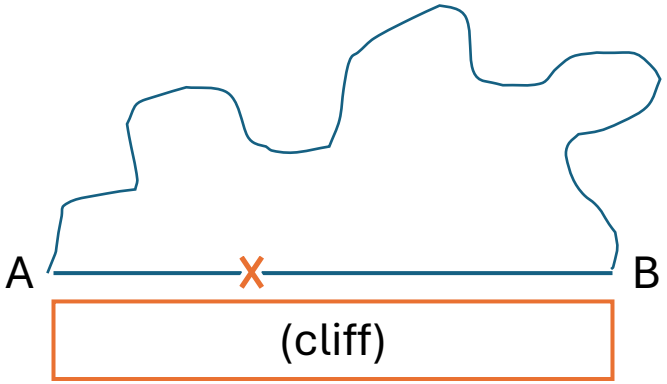
# Reinforcement Learning – Exploration Issue

- Assuming the robot aims for the optimal policy, it will naturally walk along the cliff edge
- While walking along the cliff edge, the robot might suddenly want to explore more and walk right off the cliff
- Assuming the path from A to B along the cliff edge takes 100 timesteps, the odds of getting to the other side without exploring (say, epsilon = 0.1) is near impossible



# Reinforcement Learning – Exploration Issue

- The agent still needs to explore, so how could we make the agent take the safer path?
- Sometimes we want to discourage the agent from taking any risks, only sticking to situations it knows…



# Reinforcement Learning – SARSA

- **SARSA** is very similar to Q-learning, the only difference being that it updates Q-values based on the action it *would* have taken in the future, rather than the best action it *could* have taken

**Algorithm 3** SARSA

```
Initialize Q arbitrarily.
Start in state s.
while true do
    Choose an action a.
    Execute a and observe r and s'.
    Q(s,a) = Q(s,a) + α [r + γ max_a' Q(s',a') − Q(s,a)]
    s = s'
end while
```

Only difference with Q-learning is that we remove the *max* in the update rule

In the previous example, the robot would have realized that walking along the cliff edge is too dangerous, and would instead have opted for the longer, but safer path

# Reinforcement Learning – On/Off-Policy

- Q-learning is an **Off-Policy** algorithm, while SARSA is a **On-Policy** algorithm

- **Off-Policy:** learns the optimal values as long as there is ‘enough’ exploration
  - Think of it as using two policies: one it is actively using for exploring/exploiting, and one hypothetical policy that only selects the best moves (the optimal policy)
- **On-Policy:** learns the values of the policy it is following
  - It is optimizing the same policy it is using
  - Only learns an optimal policy if selection strategy is GLIE

# Reinforcement Learning – Function Approx.

- Q-learning is great, but suffers from the **curse of dimensionality**
- Solution: **Function Approximation**

- Turn state-action pairs into features *X* and turn Q-values into labels *y*, then train a regression model to predict Q-values of state-action pairs
- Instead of a Q-table, we now have a Q-function: $Q(s, a, \omega)$ where *w* is weights

- New problem: how do we get the Q-values to train the function?

# Reinforcement Learning – Function Approx.

1. **Monte-Carlo Estimates**: unbiased but noisy. Converges to global optimum.

$$Q(s,a) = \sum \gamma^i r_i$$

2. **SARSA Estimates**: biased (each parameter is initialized randomly, so will always be off by the same amount) but less noisy. Converges close to local optimum (but will never reach optimality).

$$Q(s_t, a_t) = r_t + \gamma Q(s_{t+1}, a_{t+1})$$

3. **Q-Learning Estimates**: unstable and biased. Not guaranteed to converge.

$$Q(s_t, a_t) = r_t + \gamma \max_a Q(s_{t+1}, a)$$

# Reinforcement Learning – Policy Search

- Using values to learn a policy is great, but unstable under generalization
  - Labels change during training, samples are not independent, etc.

- (also, learning values is not really the end goal of reinforcement learning…)

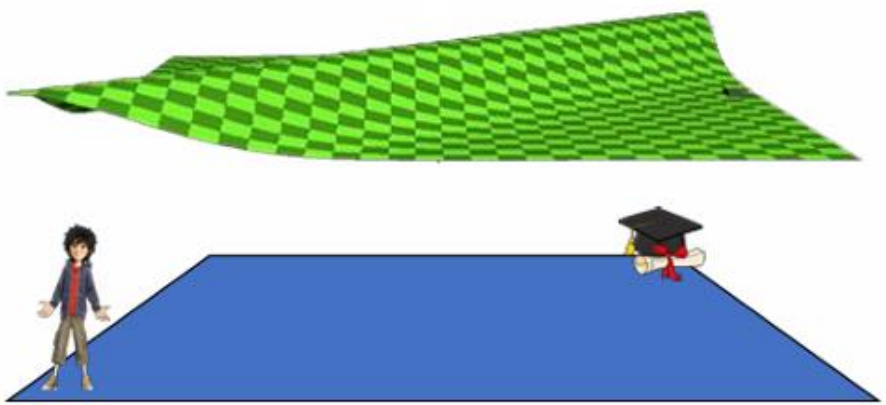
- The end goal is to learn a policy. It would be a lot easier if we could just learn the policy directly, without needing any of these intermediate steps…

# Reinforcement Learning – Ergodic MDPs

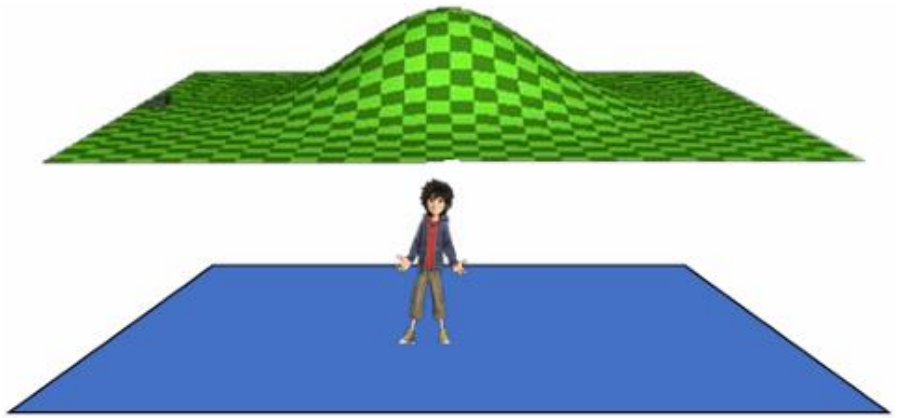
- A Markov Decision Process is **Ergodic** if:
  - Every state is often infinitely often **(recurrent)**
  - The time between visits to a state is not fixed **(aperiodic)**

If a Markov decision process is ergodic, then it has a **steady-state distribution** $d^{\pi}(s)$:

(aka, probability of agent being in that state)



An agent in an environment with a goal.



An agent in an empty environment.

# Reinforcement Learning – Policy Value

- Using the concept of a steady-state distribution, we can define a **policy value** (for continuous environments):

$$\rho(\pi) = \sum_{s} d^{\pi}(s) V^{\pi}(s) = \sum_{s,a} d^{\pi}(s) \pi(s,a) Q^{\pi}(s,a)$$

- (if the environment is episodic, we simply take the value of the starting state)

$$\rho(\pi) = V^{\pi}(s_0) \quad \text{or} \quad \rho(\pi) = \sum_{s} P(s = s_0) V^{\pi}(s)$$

# Reinforcement Learning – Policy Search

- We can find the optimal policy by maximizing over the policy value through any optimization algorithm, e.g. evolutionary algorithms:

**Algorithm 4** Evolutionary Policy Search

```
Initialize a random population of candidate policies < θ_1, θ_2, ..., θ_n >.
while not converged do
    Evaluate each policy by letting an agent run for a time T.        ▷ Fitness = Σ_{i=0}^{T} γ^i r_i
    Combine the best policies to create a new population.
end while
```

# Reinforcement Learning – Policy Gradient

- **Gradient ascent** is usually a more efficient optimization algorithm, but it requires knowledge of the **gradient** of the policy value:

$$\frac{\partial \rho(\pi_\theta)}{\partial \theta}$$

- Since the actual gradient is often unknown, we can approximate it through sampling at multiple points:

$$\frac{\partial f(\theta)}{\partial \theta} = \frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon}$$

- Through some serious rewriting we can eventually find that, to compute the gradient of a policy, we only need the **score function:**

$$\nabla_\theta \log(\pi_\theta(s, a))$$

# Reinforcement Learning – REINFORCE

- The score function can be estimated by running through the environment and collecting data. The most famous algorithm for this is **REINFORCE:**

**Algorithm 5** REINFORCE

```
Initialize a random policy π_θ.
while not converged do                                  ▷ run around the environment
    for t = 1 to T do                                   ▷ update weights every time step
        θ = θ + α ∇_θ log(π_θ(s_t, a_t)) Q^{MC}(s_t, a_t)   ▷ gradient estimate · Monte-Carlo estimate
    end for
end while
```

# Reinforcement Learning – Policy Search

- Advantages of policy search:
  - Learn policy without having to remember what to do in every state
  - Very efficient in high-dimensional or continuous action spaces
- Disadvantages of policy search:
  - Can very easily get stuck in local optima
  - Can be slow (need to sample a lot of data)
  - Can suffer from high variance

# Reinforcement Learning – Exam Questions

- True or False:
  - Q-learning finds the optimal policy as long as the selection strategy is GLIE
  - SARSA is a model-free algorithm
  - Reinforcement learning cannot learn from non-stationary probability distributions
  - In the pseudo-code below, (A) is the actor and (B) is the critic

**Algorithm 2** Actor-Critic Policy Iteration

```
Initialize π_0 and V_0 randomly.
t = 1;
while the values are still changing do
    Update the value of the states using the value update rules.      (A)
    Update the policy using the policy improvement rule.              (B)
    t = t + 1;
end while
```

- What does *P* mean in the formula: $\pi_t(s) = \underset{a}{\operatorname{argmax}} \left[ r(s,a) + \gamma \sum_{s' \in S} P_\delta(s,a,s') V^{\pi_{t-1}}(s') \right]$

# Reinforcement Learning – Exam Questions

*Q-Learning finds the optimal policy as long as all state-action pairs are visited 'often' enough (no need for GLIE)*

- True or False:
  - Q-learning finds the optimal policy as long as the selection strategy is GLIE **- False**
  - SARSA is a model-free algorithm **- True**
  - Reinforcement learning cannot learn from non-stationary probability distributions **- True**
  - In the pseudo-code below, (A) is the actor and (B) is the critic **- False**

**Algorithm 2** Actor-Critic Policy Iteration

```
Initialize π_0 and V_0 randomly.
t = 1;
while the values are still changing do
    Update the value of the states using the value update rules.    ▷ Critic
    Update the policy using the policy improvement rule.            ▷ Actor
    t = t + 1;
end while
```

- What does *P* mean in the formula: $\pi_t(s) = \underset{a}{\operatorname{argmax}} \left[ r(s,a) + \gamma \sum_{s' \in S} P_\delta(s,a,s') V^{\pi_{t-1}}(s') \right]$

**Probability of transitioning from state *s* to *s'* after choosing action *a* according to transition function $\delta$**

# 7. AlphaGo

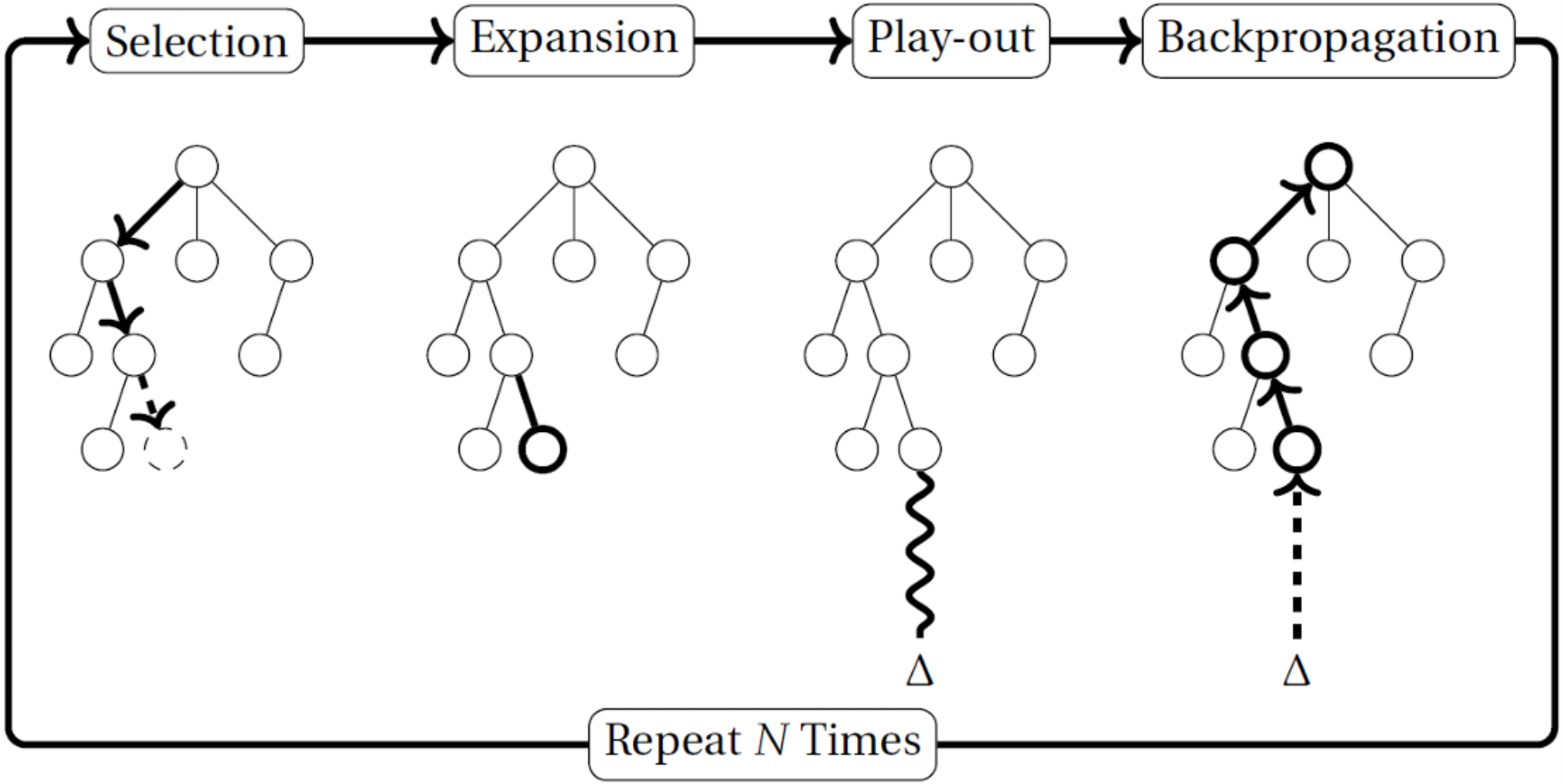
# AlphaGo – What is Go?

- State-space complexity:
  - Chess: 1x10^47
  - Go: 1.74x10^172

- Traditionally a very difficult game for computers:
  - Large branching factor
  - Difficult to create reliable heuristics
  - Human experts' knowledge is difficult to explicitly model

# AlphaGo – Monte Carlo Tree Search (MCTS)

- Traditionally the best algorithm for Go, but still nowhere near human pro's



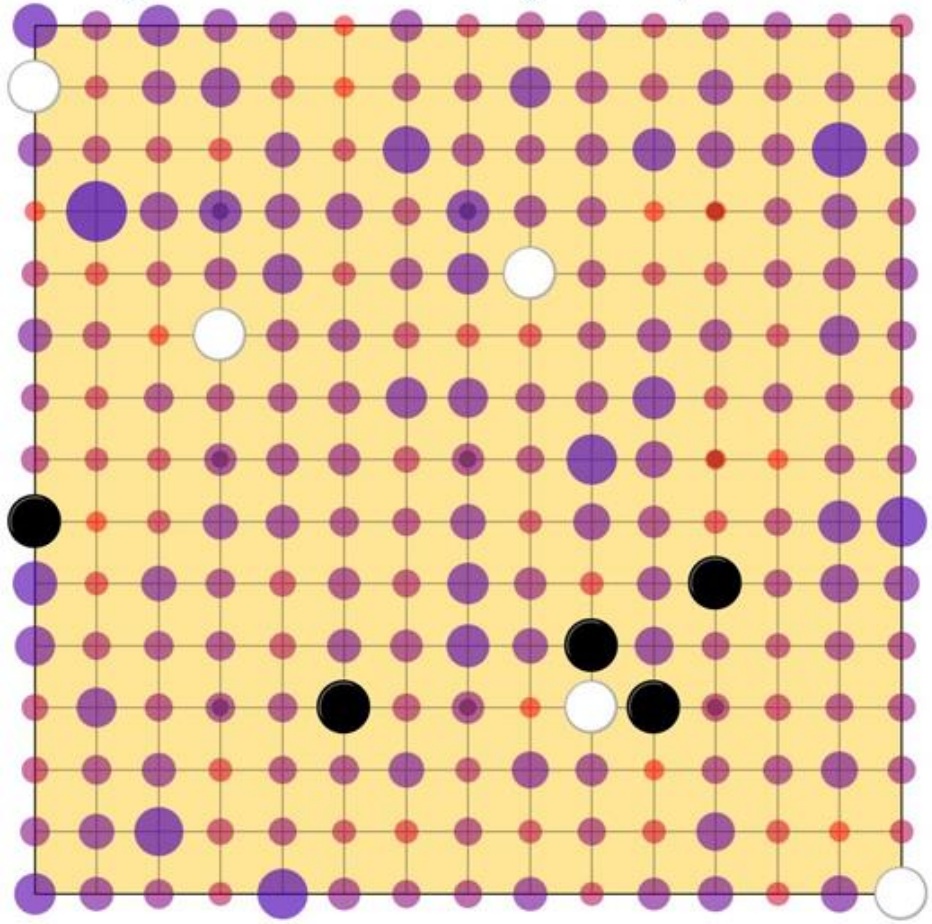
# AlphaGo – MCTS Selection

- The selection step decides which part of the tree to go down **(exploration vs exploitation)**
- Suffers from high branching factor

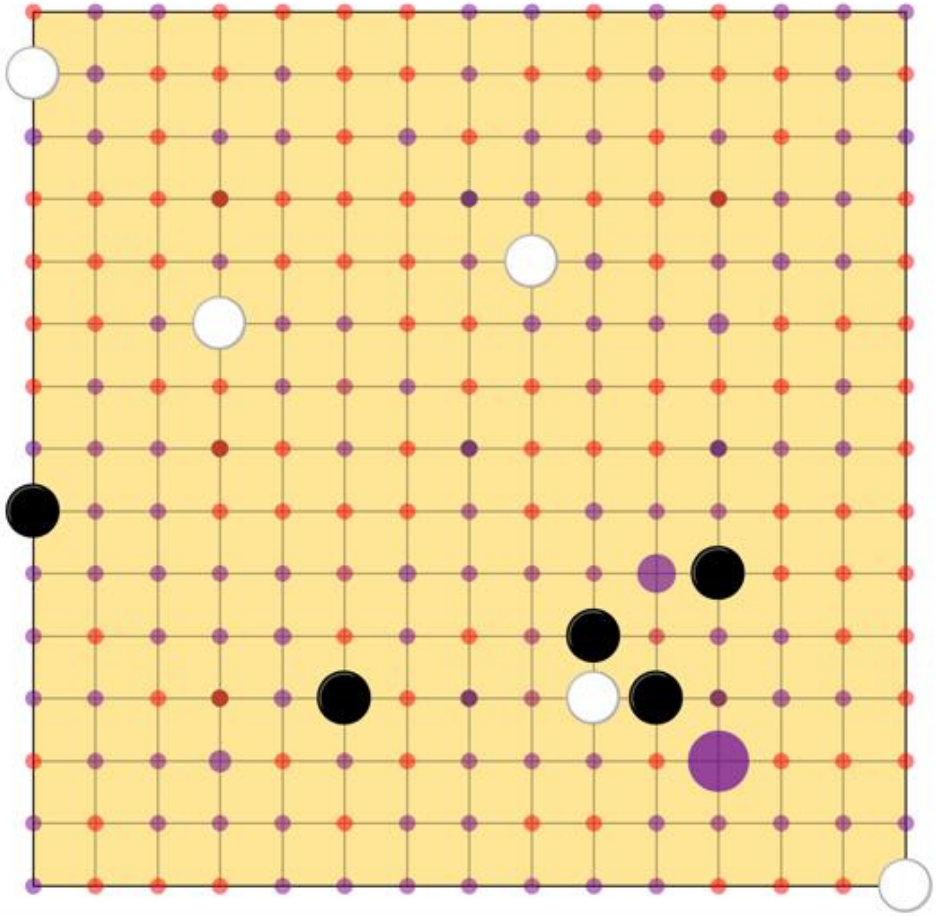
- What if we could *learn* a function that, given a state and possible actions, tells us which actions are more likely to be good (without the need of computing all possible future states)

# AlphaGo – MCTS Selection

Not guided: searching everywhere



Guided by trained function: focused search



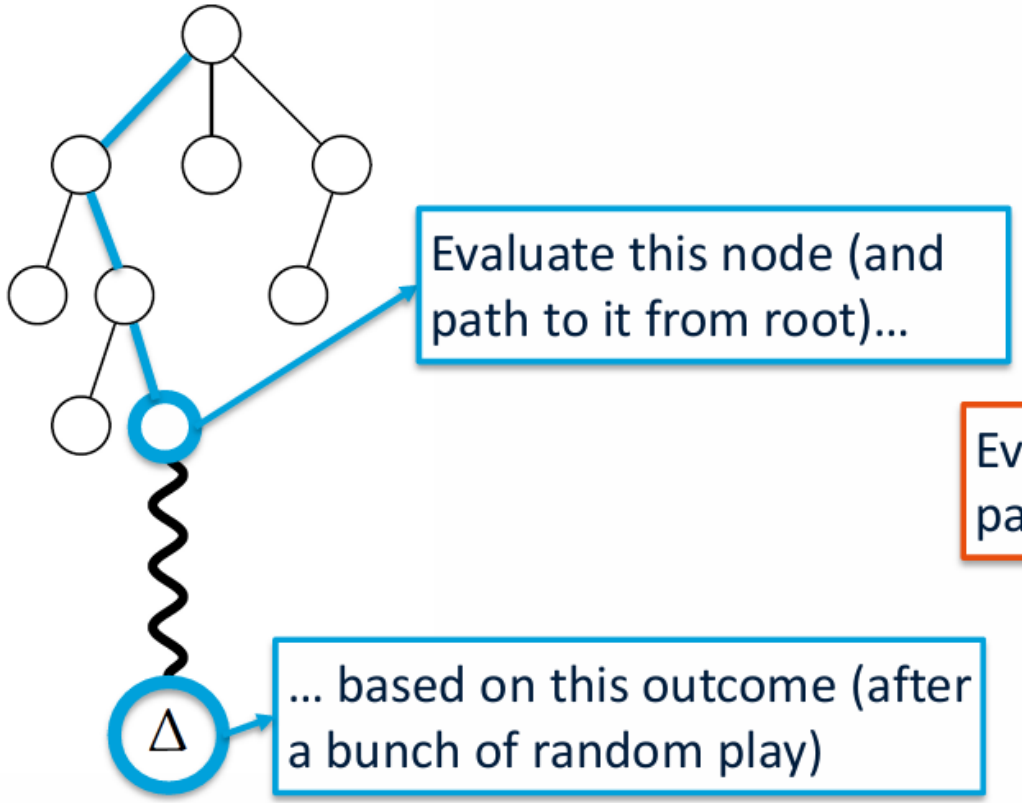
# AlphaGo – MCTS Playout + Backpropagation

- The playout step provides very noisy data for the traversed path in the tree
  - The longer the playout, the larger the noise

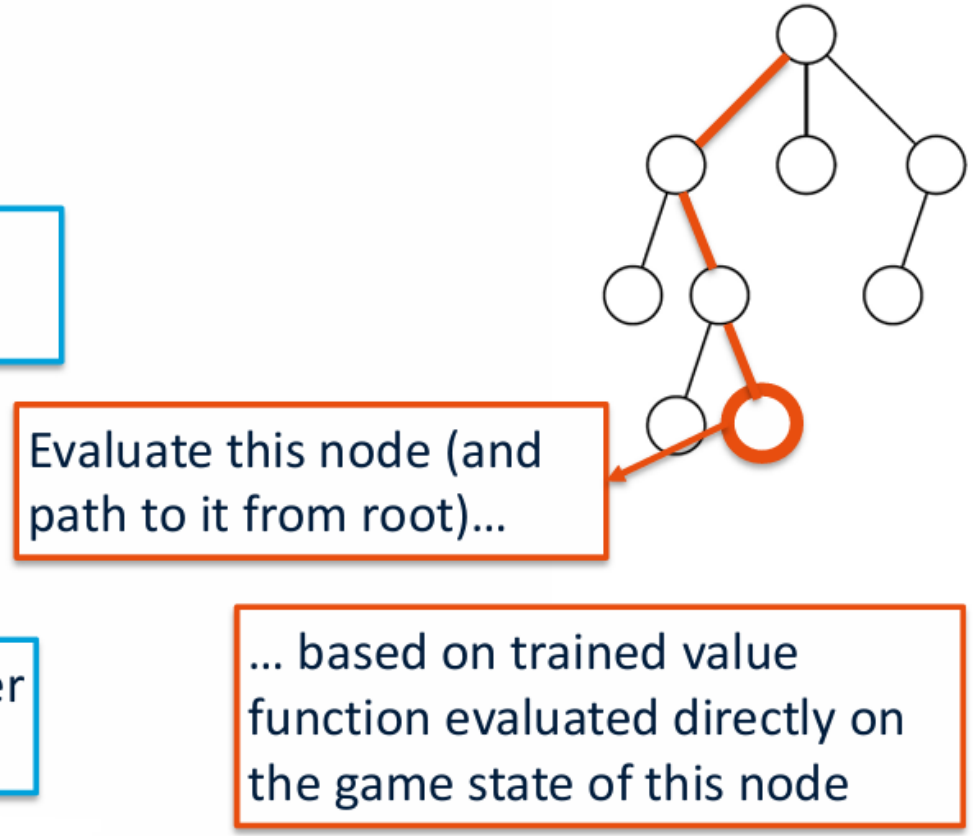
- What if we could *learn* a function that, given a non-terminal state, tells us how likely we are to win from that point onwards (without the need to simulate until we reach a terminal state)

# AlphaGo – MCTS Playout + Backpropagation

No value function




Trained value function



# AlphaGo – Machine Learning

- AlphaGo uses MCTS combined with three neural networks:

1. **Policy Network:** heatmap of ‘promising’ moves

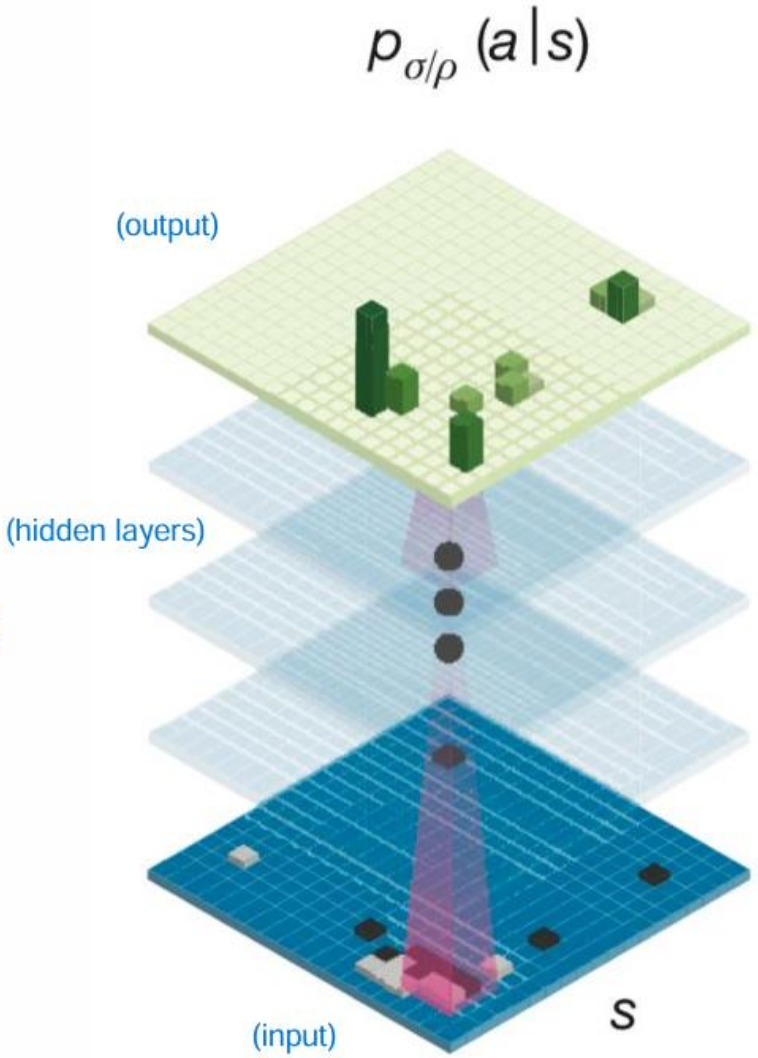
2. **Value Network:** expected outcome of a state (aka, the value function $V(s)$ )

3. **Fast Rollout Network:** smaller, faster, less accurate version of the policy network (used in simulations)

# AlphaGo – Policy Network

## Policy Network

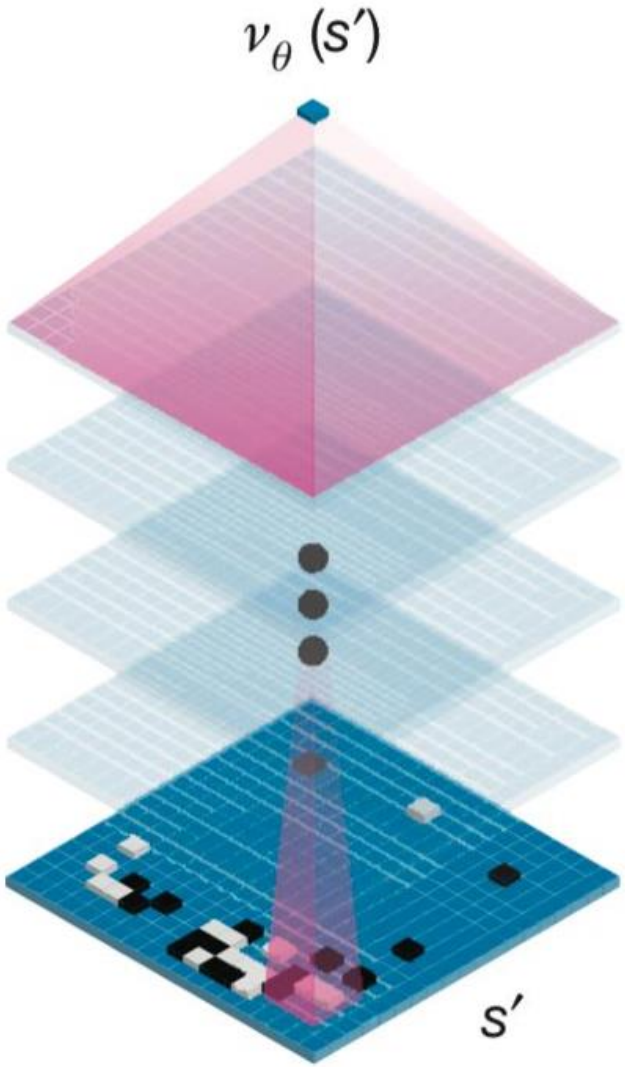
- Input:
  - Game state *s*
- Output:
  - Distribution *p* of probabilities over all legal moves



# AlphaGo – Value Network

## Value Network

- Input:
  - Game state *s'*
- Output:
  - Scalar value *v*, indicating expected outcome (0..1 = loss..win)



# AlphaGo – Selection

- Normal MCTS selection:

$$a_t = \arg\max_a Q(s_t, a) + C\sqrt{\frac{\ln(\sum_{a'} N(s_t, a'))}{N(s_t, a)}}$$

- AlphaGo MCTS selection:

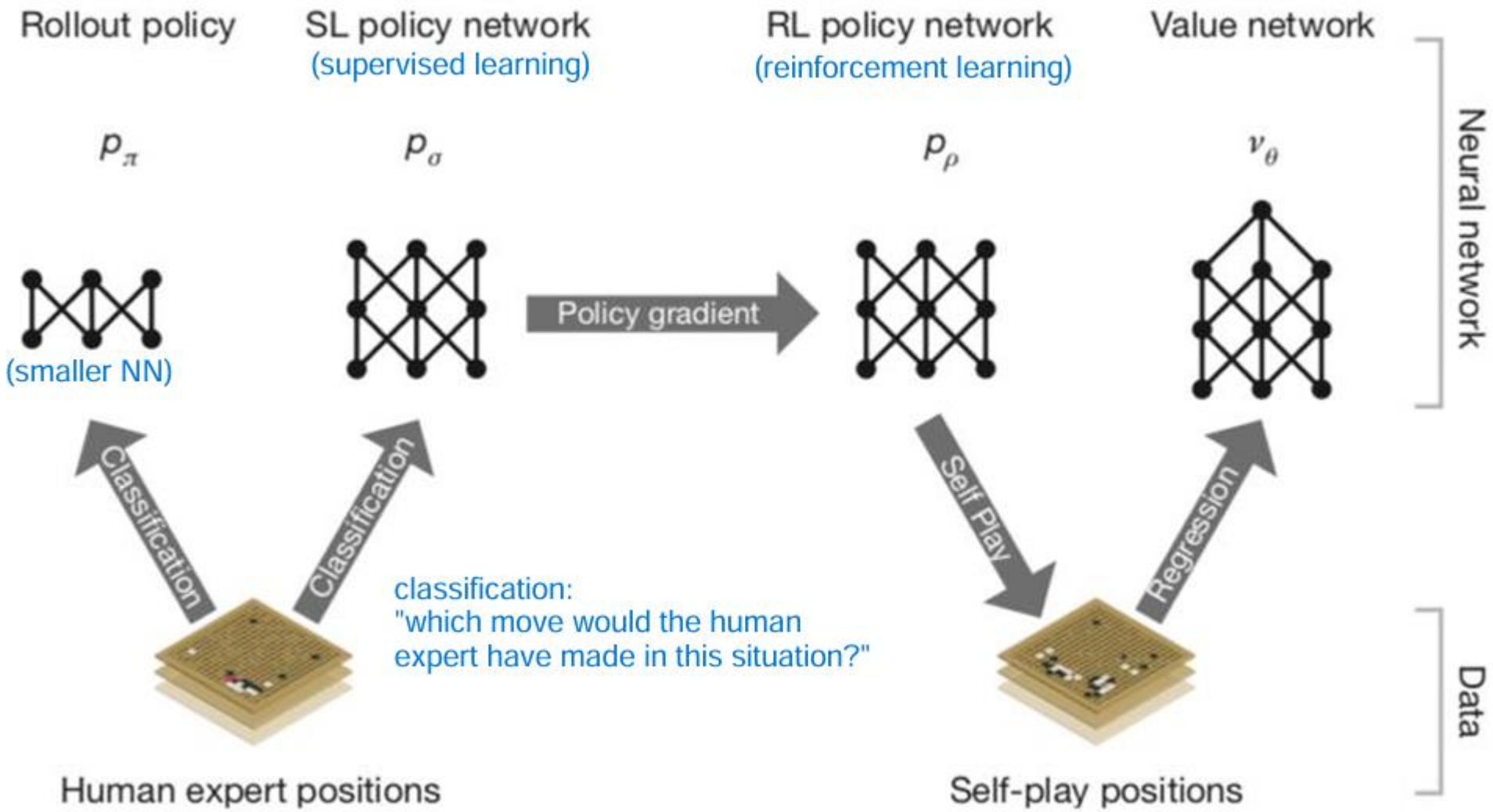
$$a_t = \arg\max_a Q(s_t, a) + C\frac{P(s_t, a)\sqrt{\sum_{a'} N(s_t, a')}}{1 + N(s_t, a)}$$

# AlphaGo – Backpropagation

- Normal MCTS backpropagation: $V(s) = \lambda z_s$
  - Where $z_s$ is the result from a randomly played out simulation

- AlphaGo MCTS backpropagation: $V(s) = (1 - \lambda)\nu_\theta(s) + \lambda z_s$
  - Where $\nu_\theta(s)$ is the value network prediction
  - Where $z_s$ is the result from a simulation played out using the fast rollout network

# AlphaGo – Training



# AlphaGo – Resources

- **Human Knowledge**: database of 160 000 games, ~30 million moves
- **Policy Network**: 50 GPUs, 3 weeks, 340 million training steps
- **Value Network**: 50 GPUs, 1 week, 50 million batches of 32 states

# AlphaGo – Results

- 18x Go world champion Lee Sedol (2016): AlphaGo won 4-1
- Go prodigy Ke Jie (2017): AlphaGo won 3-0

- One of the most famous “AI” algorithms in the world
  - Media attention, Netflix documentary, etc.
  - Before ChatGPT, probably the most common answer on the street

- Major AI “milestone”, which came much earlier than expected
- Demonstration of what is possible with Deep (reinforcement) Learning

# AlphaGo – AlphaGo Zero

- No database of human expert games
- No Go-specific features
- No simulations

- Just a single network with separate policy and value heads (outputs)
- Trained fully from self-play!

- Resources: 64 GPUs, 40 days of training, 29 million played games
- Results: outperformed AlphaGo after 72 hours of training!

# AlphaGo – AlphaZero

- Can be trained to play any game
- Outperforms humans after (on average) 24 hours of training

- Resources: 5000+ TPUs (tensor processing units, ~25million$)

- Chess: outperformed Stockfish after 4 hours (although nowadays Stockfish is better again)
- Shogi: outperformed SOTA algorithm Elmo after 2 hours
- Go: outperformed AlphaGo after 30 hours

# AlphaGo – Exam Questions

- True or False:
  - The fast rollout network has the same amount of noise as the full policy network, but is a lot faster to use in simulations
  - Self-play doesn’t learn when starting from a random policy
  - AlphaGo suffers from the high branching factor of Go
  - AlphaGo suffers from the curse of dimensionality

# AlphaGo – Exam Questions

- True or False:
  - The fast rollout network has the same amount of noise as the full policy network, but is a lot faster to use in simulations **– False** *(it is smaller, so less accurate)*
  - Self-play doesn't learn when starting from a random policy **– False** *(it does learn, just very slowly)*
  - AlphaGo suffers from the high branching factor of Go **– True**
  - AlphaGo suffers from the curse of dimensionality **– True**

# 8. Multi-Agent Reinforcement Learning

# Multi-Agent Reinforcement Learning – Why?

- Agents often have to compete or cooperate with other agents

- Multi-Agent systems are often complex, highly dynamic, nondeterministic, …
  - Difficult to design, maybe learning is a better approach?

# Multi-Agent Reinforcement Learning – What?

- What are the other agents doing? What is their plan?
  - State space *S* grows exponentially with the number of agents
- All actions jointly influence the transition and rewards
  - Action space *A* grows exponentially with the number of agents
- How to distribute rewards? Should rewards be shared or only given to the agent who performed the final action? – **Credit Assignment Problem**
  - **Single-Agent Credit Assignment Problem:** finding which actions led to the reward
  - **Multi-Agent Credit Assignment Problem:** additionally, which agents led to the reward

# MARL – Cooperation

- When agents are cooperating with each other, various rewards are possible:
- **Full System Reward:** a general reward given to all agents
- **Local Reward:** local feedback given to individual agents (quicker learning)
- **Difference Reward:** give each agent feedback about its impact (e.g. give only the difference between the reward with vs. without the agent's involvement)

# MARL – Competition

- When agents are competing with other agents, each agent usually has its own reward function

- In this scenario, agents are trying to create a **best response** policy
  - A strategy is a best response if it is optimal given the strategy of the other agents

- If all agents are playing a best response strategy, it is a **Nash Equilibrium**
  - No agent can improve by (individually) changing its policy
  - There can be multiple Nash equilibria, some better than others. An outcome where no agent can improve without making another agent worse off is a **Pareto Optimal**

# MARL – Normal Form Games

- Since actions are jointly determined by all agents, a single agent's action does not influence the environment in a fixed way anymore

- In other words, the best response policy is a moving target, which violates the Markov assumption (future is independent of past, essential for RL)

- Solution: model the environment as a **normal-form game**

# MARL – Normal Form Games

- Some examples of normal form games:

| | $C$ | $D$ |
|---|---|---|
| $C$ | $-3, -3$ | $0, -5$ |
| $D$ | $-5, 0$ | $-1, -1$ |

(a) Prisoner's Dilemma (**symmetric**)

| | $H$ | $T$ |
|---|---|---|
| $H$ | $1, -1$ | $-1, 1$ |
| $T$ | $-1, 1$ | $1, -1$ |

(b) Matching Pennies (**zero-sum**)

| | $B$ | $S$ |
|---|---|---|
| $B$ | $2, 1$ | $0, 0$ |
| $S$ | $0, 0$ | $1, 2$ |

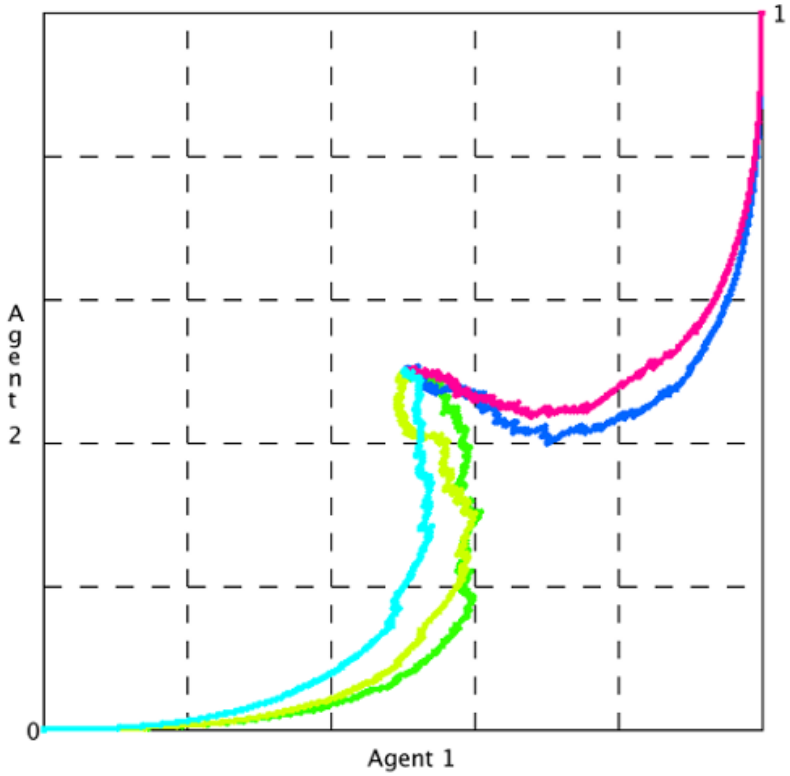
(c) Battle of the Sexes (**asymmetric**)

- These are all examples of **stateless environments**
  - Advantage: we do not need to accumulate rewards over time!

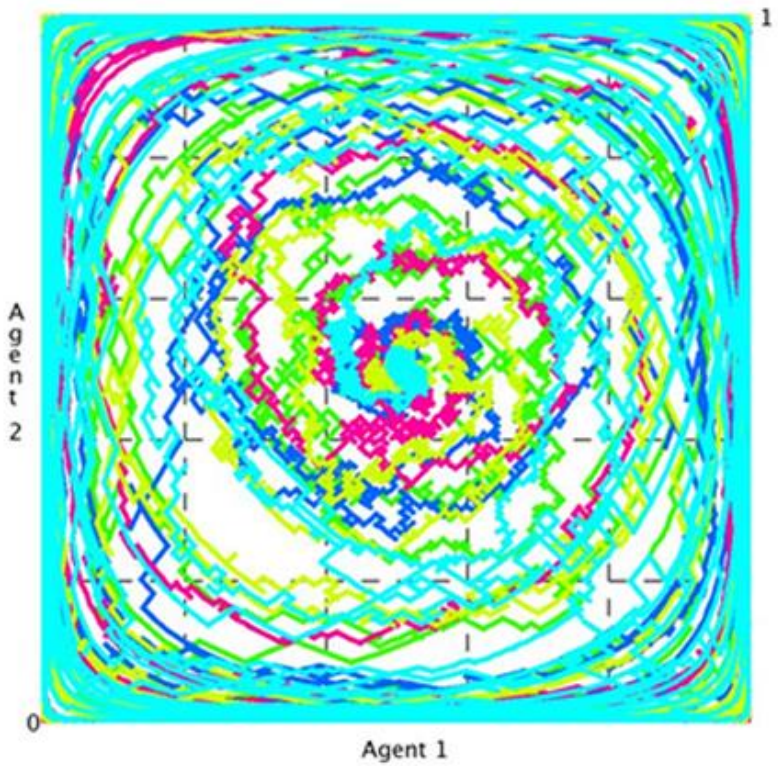
# MARL – Q-Learning

- Just pretend it’s a single-agent environment, actions from other agent are considered “noise” in the environment
- This results in partial observability, which also means that Q-learning is no longer guaranteed to converge to an optimal solution
- Still, Q-learning does work in some cases

# MARL – Q-Learning




Example of Q-learning working: Battle of the Sexes
Agents converge to Nash equilibria at (0,0) and (1,1)




Example of Q-learning not working: Matching Pennies
Agents cannot converge to mixed equilibria (0.5,0.5)

# MARL – Learning Automata

- Policy iteration for stateless environments

- No Q-values, but a vector with probabilities for each action: $\pi_t =< \pi_t(a_1), \pi_t(a_2), ..., \pi_t(a_n) >$

- Update rule is similar to Q-learning:

Reward term Penalty term

(both are scaled by action prob.)

$$\pi(a) = \pi(a) + \begin{cases} \underline{\alpha r_{t+1}(1 - \pi(a))} - \underline{\beta(1 - r_{t+1})\pi(a)} & \text{if } a = a_t \\ -\alpha r_{t+1}\pi(a) + \beta(1 - r_{t+1})\left(\frac{1}{|A|-1} - \pi(a)\right) & \text{otherwise} \end{cases}$$

Ensures action prob. still adds to 1

- Rewards must be normalized to [0, 1]!

# MARL – Network of Learning Automata

- Learning Automata were originally designed for stateless environments, but since has also been adapted for environments with states

- **Network of Learning Automata:** agent uses a separate learning automata for each state

- **State-dependent rewards:** (*T* is now, *l* is the last time this state was visited)

$$\bar{r} = \sum_{t=l+1}^{T} \frac{r_t}{T-l}$$

# MARL – Cross Learning

- Learning Automata update rule: $\pi(a) = \pi(a) + \begin{cases} \alpha r_{t+1}(1-\pi(a)) - \beta(1-r_{t+1})\pi(a) & \text{if } a = a_t \\ -\alpha r_{t+1}\pi(a) + \beta(1-r_{t+1})\left(\frac{1}{|A|-1} - \pi(a)\right) & \text{otherwise} \end{cases}$

- $\alpha, \beta \in [0,1]$ are called the **reward- and penalty parameters**

- If $\alpha = \beta$ then there is a **linear reward penalty**
- If $\beta = 0$ or $\beta = \epsilon$ there is a **linear reward inaction / linear reward e-penalty**

- Special case: **Cross Learning** $\alpha = 1$ and $\beta = 0$

# MARL – Cross Learning

- Special case: **Cross Learning** $\alpha = 1 \text{ and } \beta = 0$
- Update rule simplifies to: $\pi(a) = \pi(a) + \begin{cases} r - r\pi(a) & \text{if } a = a_t \\ -r\pi(a) & \text{otherwise} \end{cases}$
- Originates from psychology, and actually invented before learning automata
- Idea: the change in probability of the action that was executed is always positive. This means that, even if we receive a negative reward, the probability of choosing that action increases. This represents people's tendencies to repeat (possibly bad) habits. The only way to get out of a (possible bad) habit (i.e. reduce probability) is to perform a different action

# MARL – Stateless Q-Learning

- Main difference between stateless / stateful environments: agents in a stateless environment receive immediate rewards

- How to transform stateful into stateless environments? Use Q-values as immediate rewards!

- If we can find a way to compute Q-values for a stateless environments, all algorithms for stateful environments can suddenly learn in stateless environments…

# MARL – Joint Action Learning

- Aims to learn Q-values for all joint actions in a stateless environment

- Agent keeps track of all action counts for all other agents, uses this to estimate the policy of other agents:

$$E[\pi^j(a)] = \frac{C_a^j}{\sum_{b \in A^j} C_b^j}$$

Where *j* is another agent,
*a* is an action, and *C* is the count

- Its own policy is the sum of all joint action Q-values times the probability of them occurring:

$$E[V(a^i)] = \sum_{a' \in A^{-i}} Q(a') \prod_{j \neq i} E[\pi^j(a'^j)]$$

# MARL – Joint Action Learning

- Advantages:
  - It becomes possible for agents to learn how to cooperate with each other
  - Fairly straightforward implementation

- Downsides:
  - Lot of information to track, computationally expensive, etc.
  - Assumes observing other agents' actions is possible
  - Assumes the policy of other agents is static (not the case for other learning agents!)

# MARL – MiniMax Q-Learning

- Model-free reinforcement learning that learns in a stateful environment

**Algorithm 6** MiniMax-Q-Learning

Initialize $Q(s, a, o) = 1$ for all $s \in S, a \in A, o \in O$ ▷ Q-values

Opponent's actions

Initialize $V(s) = 1$ for all $s \in S$ ▷ Value function

Initialize $\pi(s, a) = \frac{1}{|A|}$ for all $s \in S, a \in A$ ▷ Initial policy

Select action $a$ with probability $\pi(s, a)$, or a random action with probability $\epsilon$

$Q(s, a, o) = (1 - \alpha)Q(s, a, o) + \alpha(r + \gamma V(s'))$ ▷ where $s'$ is the new state

$\pi = \underset{\pi'}{\operatorname{argmax}} \min_{o' \in O} \sum_{a' \in A} \pi'(s, a')Q(s, a', o')$ ▷ using linear programming

$V(s) = \min_{o' \in O} \sum_{a' \in A} \pi(s, a')Q(s, a', o')$

# MARL – MiniMax Q-Learning

- Advantages:
  - Designed for competitive zero-sum games
  - Makes few assumptions about the environment
  - Converges to a safe strategy that maximizes reward against the worst-case opponent

- Downsides:
  - Overkill for most (simple) games
  - Cannot exploit exploitable opponents

# MARL – Policy Hill Climbing

- Use two learning rates, $\alpha$ and $\delta$ to update the policy using a gradient approximation:

**Algorithm 8** Policy Hill Climbing

Initialize $Q(s,a) = 0, \pi(s,a) = \frac{1}{|A|}$ for all $s \in S, a \in A$
**while** not converged **do**
    Select (with exploration) and execute action $a$, observe $r$ and $s'$
    $Q(s,a) = (1-\alpha)Q(s,a) + \alpha(r + \gamma \max_{a'} Q(s',a'))$
    $\pi(s,a) = \pi(s,a) + \begin{cases} \delta & \text{if } a = \operatorname{argmax}_{a''} Q(s,a'') \\ \dfrac{-\delta}{|A|-1} & \text{otherwise} \end{cases}$
**end while**

# MARL – Win or Learn Fast (WoLF)

- A very popular variant of policy hill climbing

- Uses two learning rates:
  - $\delta_w$ for when it is winning
  - $\delta_l$ for when it needs to learn fast

- Compares expected value of current policy to average policy, uses winning or losing learning rate depending on if it is doing better or worse

# MARL – Win or Learn Fast (WoLF)

Number of times state *s* has been visited

1. Let $\alpha, \delta_l > \delta_w$ be learning rates. Initialize,

$$Q(s,a) \leftarrow 0, \qquad \pi(s,a) \leftarrow \frac{1}{|\mathcal{A}_i|}, \qquad C(s) \leftarrow 0.$$

2. Repeat,

(a,b) Same as PHC two slides ago

(c) Update estimate of average policy, $\bar{\pi}$,

$$\begin{aligned} C(s) &\leftarrow C(s)+1 \\ \forall a' \in \mathcal{A}_i \quad \bar{\pi}(s,a') &\leftarrow \bar{\pi}(s,a') + \\ &\quad \tfrac{1}{C(s)}\left(\pi(s,a') - \bar{\pi}(s,a')\right). \end{aligned}$$

Average selection probability of action *a* in state *s*

(d) Update $\pi(s,a)$ and constrain it to a legal probability distribution,

$$\pi(s,a) \leftarrow \pi(s,a) + \begin{cases} \delta & \text{if } a = \operatorname{argmax}_{a'} Q(s,a') \\ \frac{-\delta}{|A_i|-1} & \text{otherwise} \end{cases},$$

where,

$$\delta = \begin{cases} \delta_w & \text{if } \sum_a \pi(s,a)Q(s,a) > \sum_a \bar{\pi}(s,a)Q(s,a) \\ \delta_l & \text{otherwise} \end{cases}.$$

# MARL – Exam Questions

| | $C$ | $D$ |
|---|---|---|
| $C$ | $-3,-3$ | $0,-5$ |
| $D$ | $-5,0$ | $-1,-1$ |

(a) Prisoner's Dilemma

| | $H$ | $T$ |
|---|---|---|
| $H$ | $1,-1$ | $-1,1$ |
| $T$ | $-1,1$ | $1,-1$ |

(b) Matching Pennies

- You are in a prisoner's dilemma game with an opponent who chooses their action by flipping a (fair) coin. Describe a best reply strategy against this opponent. Is this a Nash equilibrium?
- How can a Q-learning agent converge to the (0.5,0.5) equilibrium in the matching pennies game?
  - (A) By using ε-greedy exploration
  - (B) By using Boltzmann exploration
  - (C) By using any selection strategy, as long as it explores each action often enough
  - (D) Q-Learning will never converge in the matching pennies game
- T/F: Cross learning requires the rewards to be normalized to [0,1]

# MARL – Exam Questions

| | $C$ | $D$ |
|---|---|---|
| $C$ | $-3, -3$ | $0, -5$ |
| $D$ | $-5, 0$ | $-1, -1$ |

(a) Prisoner's Dilemma

| | $H$ | $T$ |
|---|---|---|
| $H$ | $1, -1$ | $-1, 1$ |
| $T$ | $-1, 1$ | $1, -1$ |

(b) Matching Pennies

- You are in a prisoner's dilemma game with an opponent who chooses their action by flipping a (fair) coin. Describe a best reply strategy against this opponent. Is this a Nash equilibrium? **Always play C, No**

*By always playing C we get the best possible reward regardless of what our opponent does. It is not an equilibrium since our opponent can do better by also playing C. The only equilibrium in this game is (C, C)*

- How can a Q-learning agent converge to the (0.5,0.5) equilibrium in the matching pennies game?
  - (A) By using ε-greedy exploration *(greedy does not allow mixed actions)*
  - **(B) By using Boltzmann exploration** *(Boltzmann allows mixed actions, will converge with high T)*
  - (C) By using any selection strategy, as long as it explores each action often enough
  - (D) Q-Learning will never converge in the matching pennies game
- T/F: Cross learning requires the rewards to be normalized to [0,1] **- True**